ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

ĐỒ ÁN THIẾT KẾ LUẬN LÝ (CO3091)

BÁO CÁO ĐỒ ÁN

THIẾT KẾ RISC CPU DỰA TRÊN MIPS

Nhóm: L06 - 11 Học kì: 251

Giảng viên hướng dẫn: Tôn Huỳnh Long

Sinh viên:	Huỳnh Ngọc Yến Nhi	2312497
	Nguyễn Anh Thư	2313385
	Phan Ngọc Phương Trinh	2313582

TP. Hồ Chí Minh, 2026

Mục lục

1 Giới thiệu đề tài **1**

2 Mục tiêu đề tài **1**

3 Cơ sở lý thuyết **1**

3.1 Kiến trúc RISC VÀ MIPS 1

3.2 Ứng dụng thực tiễn 2

4 Thiết kế và hiện thực **5**

4.1 Tổng quan hệ thống 5

4.2 Thiết kế và hiện thực Instruction Fetch 7

4.3 Thiết kế và hiện thực Instruction Decode 13

4.4 Thiết kế và hiện thực Execute 34

4.5 Thiết kế và hiện thực Memory Access 45

4.6 Thiết kế và hiện thực Write Back 48

4.7 Thiết kế và hiện thực Pipeline Registers 51

5 Kiểm thử và đánh giá **60**

5.1 Kiểm thử 60

5.2 Đánh giá hiệu năng hệ thống 74

6 Tổng kết **78**

6.1 Vivado Project Source Code 78

6.2 Kết luận 78

6.3 Hạn chế và hướng phát triển 79

1 Giới thiệu đề tài

Bộ xử lý **RISC (Reduced Instruction Set Computer)** là môt mô hình có cấu trúc rõ ràng và dễ triển khai, thường được sử dụng để nghiên cứu nguyên lý hoạt động của CPU. Kiến trúc này nổi bật với tập lệnh đơn giản, định dạng lệnh thống nhất và luồng xử lý dữ liệu mạch lạc, phù hợp cho việc thiết kế và hiện thực các bộ xử lý cơ bản.

Đề tài **"Thiết kế RISC CPU 16-bit dựa trên kiến trúc MIPS"** tập trung vào việc xây dựng một bộ xử lý RISC với độ rộng dữ liệu 16-bit nhằm giảm độ phức tạp nhưng vẫn đảm bảo đầy đủ các khối chức năng chính của một CPU. Hệ thống được hiện thực bằng ngôn ngữ mô tả phần cứng **Verilog**, bao gồm các khối như nạp lệnh, giải mã lệnh, thực thi, truy cập bộ nhớ và ghi kết quả.

Thông qua việc thiết kế, mô phỏng và kiểm tra hoạt động của CPU, đề tài giúp làm rõ luồng xử lý của kiến trúc MIPS cũng như mối liên hệ giữa các khối chức năng trong một bộ xử lý **RISC** hoàn chỉnh.

2 Mục tiêu đề tài

Mục tiêu của đề tài là thiết kế và hiện thực một bộ xử lý RISC CPU dựa trên kiến trúc MIPS 16-bit dựa trên các kiến thức về thiết kế luận lý với HDL và kiến trúc máy tính. Bộ xử lý được mô tả bằng ngôn ngữ Verilog, đảm bảo thể hiện đầy đủ các khối chức năng cơ bản của một CPU theo kiến trúc MIPS.

Bên cạnh đó, đề tài hướng đến việc mô phỏng, kiểm tra và đánh giá hoạt động của CPU nhằm đảm bảo tính đúng đắn của thiết kế theo đặc tả đã đề ra. Kết quả thiết kế có khả năng triển khai trên kit FPGA Arty-Z7 hoặc các nền tảng tương đương, đáp ứng các yêu cầu kỹ thuật trong tài liệu hướng dẫn.

3 Cơ sở lý thuyết

3.1 Kiến trúc RISC VÀ MIPS

3.1.1 RISC (Reduced Instruction Set Computer)

Kiến trúc RISC (Reduced Instruction Set Computer) là một triết lý thiết kế bộ vi xử lý tập trung vào việc đơn giản hóa tập lệnh nhằm tăng hiệu suất và giảm độ phức tạp của phần cứng. Kiến trúc này phù hợp cho việc thiết kế các bộ xử lý cơ bản và là nền tảng của nhiều kiến trúc CPU hiện đại, trong đó có MIPS.

Các nguyên tắc cốt lõi của kiến trúc RISC bao gồm:

- Tập lệnh đơn giản, số lượng lệnh hạn chế và dễ thực hiện.
- Mỗi lệnh được thiết kế để thực thi trong một chu kỳ máy hoặc thuận lợi cho việc xây dựng pipeline.
- Chỉ các lệnh `LOAD` và `STORE` mới được phép truy cập bộ nhớ, các phép toán còn lại thực hiện trực tiếp trên thanh ghi.
- Số lượng thanh ghi lớn nhằm giảm số lần truy cập bộ nhớ, từ đó cải thiện hiệu năng.
- Định dạng lệnh cố định, giúp quá trình giải mã lệnh trở nên đơn giản và nhanh chóng.

Dựa trên các nguyên tắc trên, kiến trúc RISC được lựa chọn làm nền tảng cho thiết kế CPU 16-bit trong đề tài. Phiên bản 16-bit giúp giảm độ phức tạp của phần cứng trong khi vẫn giữ được cấu trúc và luồng xử lý đặc trưng của kiến trúc RISC, phù hợp cho việc thiết kế, mô phỏng và kiểm tra hoạt động của bộ xử lý.

3.1.2 MIPS (Microprocessor without Interlocked Pipeline Stages)

MIPS là một kiến trúc RISC điển hình được phát triển bởi John Hennessy tại Đại học Stanford. Kiến trúc MIPS 32-bit truyền thống có 32 thanh ghi đa năng, sử dụng ba định dạng lệnh chính là R-type, I-type và J-type, và áp dụng pipeline 5 giai đoạn bao gồm IF (Instruction Fetch), ID (Instruction Decode), EX (Execute), MEM (Memory Access) và WB (Write Back).

MIPS 16-bit là phiên bản thu gọn của MIPS 32-bit, được tối ưu hóa cho các ứng dụng nhúng và hệ thống tài nguyên hạn chế. Kiến trúc này sử dụng thanh ghi và bus dữ liệu 16-bit, cho phép truy cập không gian bộ nhớ lên đến 64KB. Số lượng thanh ghi được giảm xuống để phù hợp với độ rộng lệnh 16-bit. Định dạng lệnh trong MIPS 16-bit được điều chỉnh để phù hợp với giới hạn 16-bit, bao gồm R-type cho các phép toán giữa các thanh ghi, I-type cho phép toán với hằng số và truy cập bộ nhớ, và J-type cho các lệnh nhảy.

3.2 Ứng dụng thực tiễn

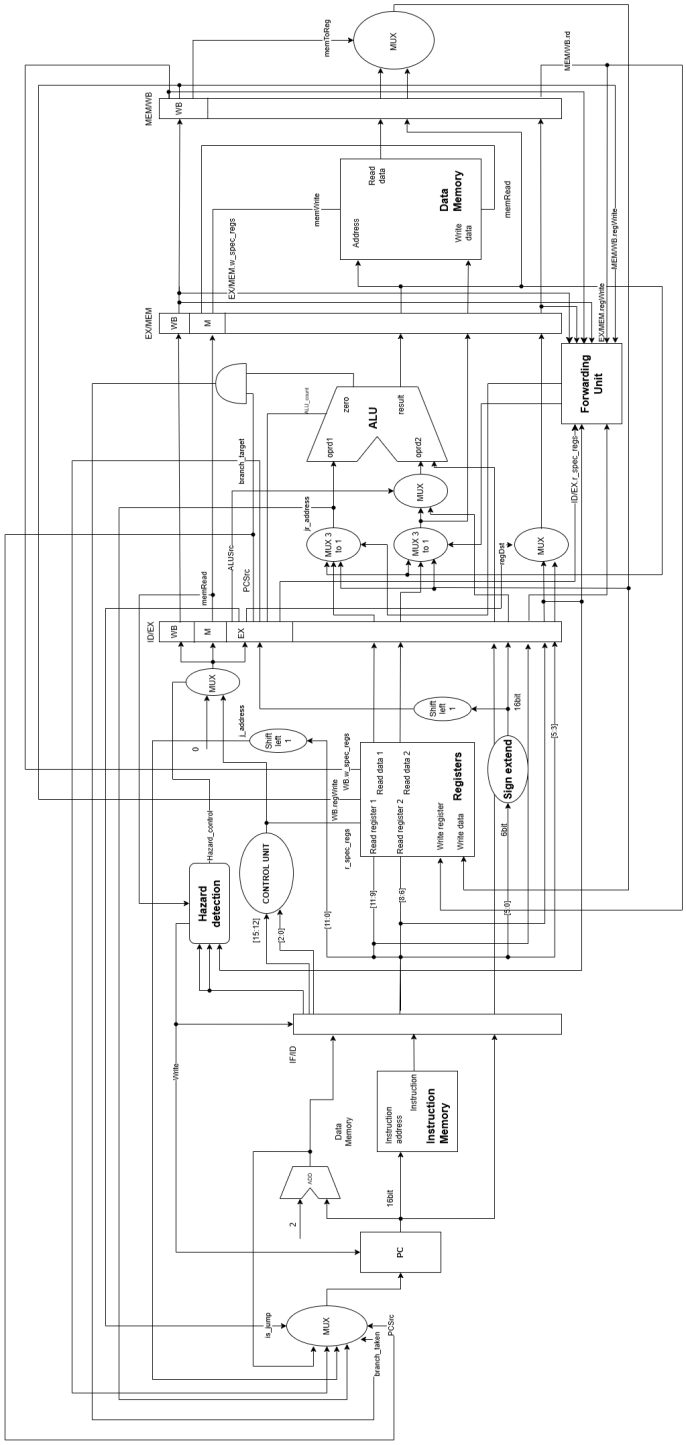
Thiết kế CPU RISC 16-bit có nhiều ứng dụng thực tiễn trong các lĩnh vực khác nhau nhờ kiến trúc đơn giản, linh hoạt và dễ triển khai.

- **Giáo dục và đào tạo**: RISC 16-bit là công cụ hiệu quả để minh họa các khái niệm cơ bản của kiến trúc máy tính và thiết kế CPU như datapath, control unit và pipeline, giúp người học hiểu rõ nguyên lý hoạt động của bộ xử lý ở mức phần cứng.

- **Hệ thống nhúng đơn giản**: Kiến trúc RISC 16-bit phù hợp cho các thiết bị nhúng và IoT có yêu cầu xử lý không quá phức tạp, đáp ứng tốt các tiêu chí về tiêu thụ điện năng thấp và chi phí triển khai hợp lý.
- **Triển khai trên FPGA**: Bộ xử lý RISC 16-bit có thể được triển khai trên các nền tảng FPGA với tài nguyên hạn chế, cho phép xây dựng các bộ xử lý tùy chỉnh phục vụ những ứng dụng chuyên biệt.
- **Nguyên mẫu và nghiên cứu**: Thiết kế RISC 16-bit là nền tảng thuận lợi cho việc thử nghiệm và đánh giá các kỹ thuật như pipeline, cache hoặc xử lý xung đột, trước khi áp dụng vào các kiến trúc phức tạp hơn.
- **Vi điều khiển và thiết bị điều khiển**: Kiến trúc này có thể được sử dụng làm cơ sở phát triển các vi điều khiển và thiết bị đo lường, giám sát và điều khiển tự động, nhờ khả năng xử lý ổn định và độ trễ thấp.
- **Phát triển công cụ lập trình**: MIPS 16-bit cung cấp nền tảng phù hợp để xây dựng các công cụ phần mềm như assembler, simulator hoặc compiler, phục vụ nghiên cứu và phát triển phần mềm cấp thấp.

4 Thiết kế và hiện thực

4.1 Tổng quan hệ thống



Hình 4.1: *Sơ đồ hệ thống*

Sơ đồ trình bày kiến trúc tổng thể của bộ vi xử lý RISC 16-bit với pipeline 5 giai đoạn, bao gồm các khối chức năng chính và đường truyền dữ liệu giữa chúng. Hệ thống được thiết kế theo nguyên tắc Harvard architecture với bộ nhớ lệnh và bộ nhớ dữ liệu tách biệt, đồng thời tích hợp các cơ chế xử lý hazard và forwarding để đảm bảo hiệu suất pipeline. Với các khối chức năng chính:

- Instruction Fetch (IF): Khối IF bao gồm Program Counter (PC), Instruction Memory và các mạch logic tính toán địa chỉ lệnh tiếp theo. PC được cập nhật mỗi chu kỳ thông qua MUX lựa chọn giữa PC+2 (lệnh tuần tự), địa chỉ branch, hoặc địa chỉ jump. Instruction Memory nhận địa chỉ 16-bit từ PC và xuất ra lệnh 16-bit tương ứng.
- Instruction Decode (ID): Giai đoạn ID thực hiện giải mã lệnh và đọc dữ liệu từ thanh ghi. Control Unit nhận opcode và function code để sinh ra các tín hiệu điều khiển cho toàn bộ datapath. Register File chứa 8-16 thanh ghi, với hai cổng đọc (Read register 1, Read register 2) xuất dữ liệu đồng thời và một cổng ghi (Write register) để ghi kết quả về. Khối Sign Extend mở rộng trường immediate từ 6-bit lên 16-bit với bảo toàn dấu. Hazard Detection Unit giám sát các xung đột dữ liệu.
- Execute (EX): Khối EX là trung tâm xử lý số học và logic của CPU. ALU thực hiện các phép toán dựa trên tín hiệu điều khiển ALU count nhận từ Control Unit. Các MUX ở đầu vào ALU lựa chọn giữa dữ liệu từ thanh ghi, giá trị immediate, hoặc dữ liệu forwarding từ các giai đoạn sau. Output zero từ ALU được sử dụng để quyết định điều kiện branch.
- Memory Access (MEM): Giai đoạn MEM xử lý truy cập Data Memory cho các lệnh load và store. Data Memory có cổng Address nhận địa chỉ từ kết quả ALU, cổng Write data nhận dữ liệu cần ghi (từ thanh ghi nguồn), và cổng Read data xuất dữ liệu đọc được. Tín hiệu điều khiển memWrite và memRead quyết định thao tác ghi hoặc đọc. Khối này cũng chuyển tiếp kết quả ALU và dữ liệu đọc được sang giai đoạn WB.
- Write Back (WB): Giai đoạn WB quyết định nguồn dữ liệu sẽ được ghi về Register File. Tín hiệu regWrite cho phép ghi dữ liệu vào thanh ghi đích được chỉ định.

Ngoài ra còn có các cơ chế hỗ trợ pipeline như:

- Forwarding Unit: Forwarding Unit giải quyết data hazard bằng cách chuyển tiếp dữ liệu từ giai đoạn MEM hoặc WB trực tiếp về giai đoạn EX mà không cần chờ ghi vào Register File. Unit này so sánh địa chỉ thanh ghi nguồn đang cần ở EX với địa chỉ thanh ghi đích ở MEM và WB, sau đó điều khiển các MUX để chọn đúng nguồn dữ liệu.
- Hazard Detection Unit: Hazard Detection Unit phát hiện các tình huống load-use hazard khi lệnh load chưa hoàn thành nhưng lệnh tiếp theo đã cần dùng dữ liệu. Trong trường hợp này, unit phát

tín hiệu stall để dừng pipeline một chu kỳ, đồng thời chèn bubble (NOP) vào giai đoạn EX.

- Pipeline Registers: Các thanh ghi pipeline (IF/ID, ID/EX, EX/MEM, MEM/WB) lưu trữ dữ liệu và tín hiệu điều khiển giữa các giai đoạn, đảm bảo tất cả giai đoạn hoạt động đồng bộ theo xung clock. Mỗi thanh ghi pipeline hoạt động như một buffer, cho phép nhiều lệnh thực thi đồng thời ở các giai đoạn khác nhau.

4.2 Thiết kế và hiện thực Instruction Fetch

Giai đoạn Instruction Fetch (IF) là giai đoạn đầu tiên trong pipeline, có nhiệm vụ lấy lệnh từ bộ nhớ chương trình dựa trên giá trị của Program Counter (PC). Giai đoạn này quyết định địa chỉ lệnh tiếp theo sẽ thực thi, xử lý các trường hợp nhảy (jump), rẽ nhánh (branch), và thực thi tuần tự. Với các module thành phần:

4.2.1 Program Counter (PC)

Module Program Counter (PC) là một thanh ghi 16-bit dùng để lưu trữ địa chỉ của lệnh hiện tại đang được thực thi. PC được cập nhật ở mỗi chu kỳ clock dựa trên các tín hiệu điều khiển của hệ thống.

Tín hiệu	Độ rộng	Hướng	Chức năng
clk	1	Input	Xung clock đồng bộ hệ thống.
rst	1	Input	Tín hiệu reset, khi được kích hoạt sẽ đưa PC về giá trị `0x0000`.
halt	1	Input	Tín hiệu dừng CPU, ngăn không cho PC cập nhật.
write	1	Input	Tín hiệu cho phép ghi từ Hazard Detection Unit, dùng để stall pipeline.
pc_next	16	Input	Giá trị PC tiếp theo được chọn từ multiplexer.
pc	16	Output	Địa chỉ lệnh hiện tại (16-bit).

Bảng 1: *Các tín hiệu của module Program Counter*

Nguyên lý hoạt động: PC được khởi tạo bằng `0x0000` khi tín hiệu reset được kích hoạt. Ở mỗi cạnh lên của clock, nếu không có tín hiệu `halt` và tín hiệu `write` được cho phép, PC sẽ cập nhật giá trị mới từ `pc_next`. Cơ chế này cho phép hệ thống dừng pipeline khi phát hiện hazard hoặc khi CPU bị halt.

Hiện thực logic chính:

```
    initial begin
        pc <= 16'h0000;
    end
    always @(posedge clk or posedge rst) begin
        if (rst) begin
```

```
            pc <= 16'h0000;
        end
        else if (!halt && write) begin
            pc <= pc_next;
        end
```

4.2.2 Adder PC (ADD_PC)

Module ADD_PC thực hiện phép cộng để tính toán địa chỉ lệnh kế tiếp trong trường hợp thực thi tuần tự. Do mỗi lệnh có độ dài 16-bit (2 byte), địa chỉ tiếp theo được xác định bằng cách cộng thêm 2 vào giá trị PC hiện tại.

Tín hiệu	Độ rộng	Hướng	Chức năng
pc	16	Input	Giá trị Program Counter hiện tại
pc_plus	16	Output	Giá trị PC tăng tuần tự (PC + 2)

Bảng 2: *Các tín hiệu của module ADD_PC*

Nguyên lý hoạt động: ADD_PC là module tổ hợp, sử dụng phép gán trực tiếp để tính toán `pc_plus = pc + 2`. Module không sử dụng clock và không gây trễ, đảm bảo địa chỉ kế tiếp luôn sẵn sàng cho các khối xử lý tiếp theo.

Hiện thực logic chính:

```
    assign pc_plus = pc + 16'd2;
```

4.2.3 Multiplexer (MUX_PC)

Module MUX_PC là bộ chọn nguồn địa chỉ cho PC tiếp theo, đóng vai trò quyết định luồng điều khiển chương trình trong giai đoạn Instruction Fetch.

Tín hiệu	Độ rộng	Hướng	Chức năng
pc_plus	16	Input	Địa chỉ tuần tự tiếp theo (PC + 2)
address_target_id	16	Input	Địa chỉ đích từ giai đoạn ID cho lệnh jump.
address_target_ex	16	Input	Địa chỉ đích từ giai đoạn EX cho lệnh branch.
jr_address	16	Input	Địa chỉ lấy từ thanh ghi cho lệnh Jump Register.
PCSrc_ex	2	Input	Tín hiệu điều khiển 2-bit từ giai đoạn EX.
is_jump	1	Input	Tín hiệu điều khiển jump.
branch_taken	1	Input	Tín hiệu báo điều kiện branch được thỏa mãn.
pc_next	16	Output	Địa chỉ PC tiếp theo được lựa chọn.

Bảng 3: *Các tín hiệu của module MUX_PC*

Nguyên lý hoạt động: MUX_PC lựa chọn địa chỉ PC tiếp theo theo thứ tự ưu tiên như sau:

1. **Jump**: Khi `is_jump = 1`, chọn `address_target_id`.
2. **Jump Register**: Khi `PCSrc_ex = 01`, chọn `jr_address`.
3. **Branch**: Khi `branch_taken = 1`, chọn `address_target_ex`.
4. **Tuần tự**: Trường hợp mặc định, chọn `pc_plus`.

Hiện thực logic chính:

```
always @(*) begin
    if (is_jump === 1'b1) begin
        if (^address_target_id !== 1'bx && ^address_target_id !==
            1'bz) begin
            pc_next = address_target_id;
        end
        else begin
            $display("[MUX_PC] WARNING: address_target_id=0x%04h is
                X/Z!",
                    address_target_id);
        end
    end
    else if (PCSrc_ex == 2'b01) begin
        if (^jr_address !== 1'bx && ^jr_address !== 1'bz) begin
            pc_next = jr_address;
```

```
                $display(" address = %h", jr_address);
            end
            else $display(" ERROR ADDRESS %h", jr_address);
        end
        else if (branch_taken == 1'b1) begin
            if (^address_target_ex !== 1'bx && ^address_target_ex !==
                1'bz) begin
                pc_next = address_target_ex;
            end
            if (branch_taken) begin
            $display("[MUX_PC] Time=%0t BRANCH! address_target_ex=0x%04
                h pc_next=0x%04h",
                    $time, address_target_ex, pc_next);
            end
        end else if (^pc_plus === 1'bx || ^pc_plus === 1'bz) begin
            pc_next = 16'h0002;
            $display("[MUX_PC] WARNING: pc_plus is X/Z!");
        end
        else begin
            pc_next = pc_plus;
        end
    end
```

4.2.4 Instruction Memory (IM)

Instruction Memory (IM) là bộ nhớ chỉ đọc (ROM) dùng để lưu trữ các lệnh của chương trình. Nội dung bộ nhớ được khởi tạo từ file `program.mem`, là file đã được chuẩn bị sẵn và tích hợp trực tiếp trong source project Vivado, và được nạp vào bộ nhớ khi bắt đầu quá trình mô phỏng.

Tín hiệu	Độ rộng	Hướng	Chức năng
addr	16	Input	Địa chỉ byte từ PC (16-bit).
instr	16	Output	Lệnh 16-bit tương ứng với địa chỉ được yêu cầu.

Bảng 4: *Các tín hiệu của module Instruction Memory*

Nguyên lý hoạt động: IM chuyển đổi địa chỉ byte sang địa chỉ word bằng cách dịch phải một bit. Nếu

địa chỉ hợp lệ trong phạm vi cho phép, module trả về lệnh tương ứng. Trường hợp địa chỉ không hợp lệ, module trả về mã lệnh `0xC000` (NOP). Dữ liệu bộ nhớ được nạp bằng lệnh `$readmemh` khi khởi tạo mô phỏng.

Hiện thực logic chính:

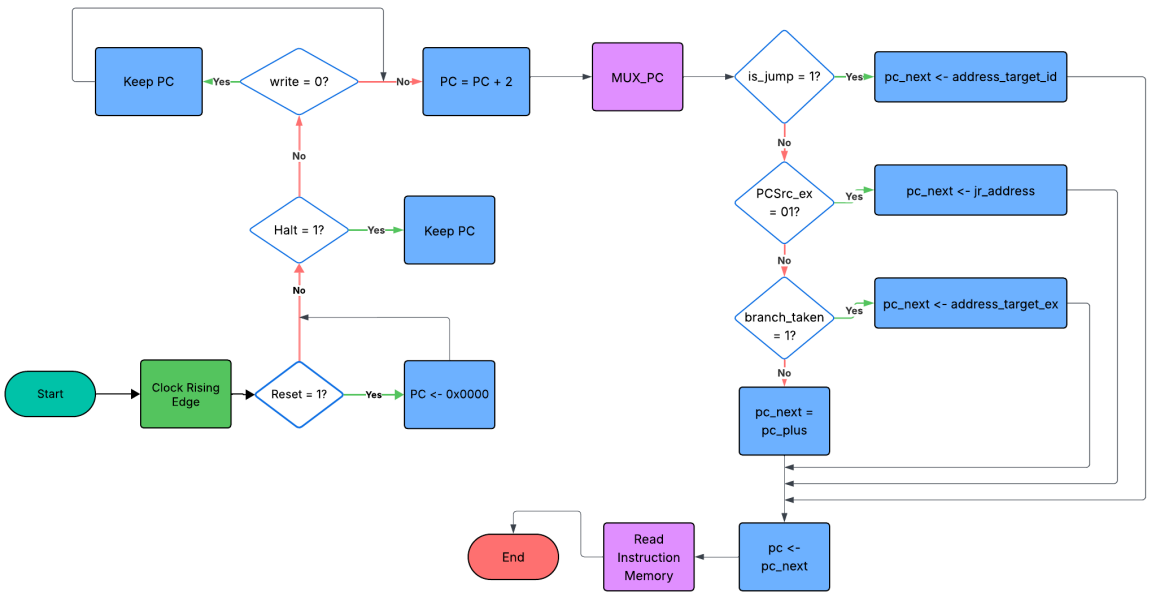
```
reg [DATA_WIDTH-1:0] memory [0:MEM_SIZE-1];

wire [ADDR_WIDTH-2:0] word_addr = addr[ADDR_WIDTH-1:1];

always @(*) begin
    instr = 16'C0000;

    if (word_addr < MEM_SIZE)
        instr = memory[word_addr];
end

integer i;
initial begin
    if (INIT_FILE != "") begin
        $readmemh(INIT_FILE, memory);
        $display("[IM] Loaded %s into instruction memory",
            INIT_FILE);
    end else begin
        for (i = 0; i < MEM_SIZE; i = i + 1)
            memory[i] = 16'hC000;
    end
end
```

4.2.5 Bảng tín hiệu IF Stage (Interface)

Tên tín hiệu	Loại	Độ rộng	Mô tả
Clock và Reset			
clk	Input	1-bit	Tín hiệu clock đồng bộ hệ thống
rst	Input	1-bit	Reset, đặt PC về 0x0000 (active high)
Control Signals			
halt	Input	1-bit	Dừng CPU, ngăn cập nhật PC
pc_write	Input	1-bit	Cho phép ghi PC (từ Hazard Detection), 0: stall
is_jump	Input	1-bit	Báo lệnh jump (J, JAL) từ ID
branch_taken	Input	1-bit	Báo branch được thực hiện từ EX
EX_PCSrc	Input	2-bit	Nguồn PC từ EX (00: sequential, 01: JR)
Address Inputs			
address_target_id	Input	16-bit	Địa chỉ đích cho lệnh jump từ ID
address_target_ex	Input	16-bit	Địa chỉ đích cho lệnh branch từ EX
jr_address	Input	16-bit	Địa chỉ thanh ghi cho lệnh JR từ EX
Outputs			
pc_current	Output	16-bit	Địa chỉ lệnh hiện tại
pc_plus_2	Output	16-bit	Địa chỉ lệnh tiếp theo (PC + 2)
instruction	Output	16-bit	Lệnh được fetch từ Instruction Memory

Bảng 5: *Bảng tín hiệu IF Stage*

4.2.6 Luồng xử lý dữ liệu



Hình 4.2: *Flowchart mô tả luồng hoạt động*

Có thể thấy từ mã nguồn nhóm đã thực hiện và luồng dữ liệu trong hình vẽ, PC chỉ bắt đầu hoạt động sau khi tín hiệu reset được kích hoạt và sau đó được đưa về trạng thái không kích hoạt. Do đó, trong quá trình kiểm thử, tín hiệu reset cần được tác động đúng trình tự để đảm bảo PC và toàn bộ hệ thống hoạt động đúng như thiết kế.

4.3 Thiết kế và hiện thực Instruction Decode

Giai đoạn *Instruction Decode* (ID) là giai đoạn trung gian trong kiến trúc pipeline, đóng vai trò cầu nối giữa giai đoạn nạp lệnh (Instruction Fetch) và các giai đoạn thực thi tiếp theo. Nhiệm vụ chính của ID stage là phân tích cấu trúc lệnh, sinh tín hiệu điều khiển và chuẩn bị dữ liệu cần thiết cho quá trình thực thi lệnh.

Cụ thể, giai đoạn ID thực hiện các chức năng sau:

- Phân tích cấu trúc lệnh để xác định loại lệnh, các thanh ghi nguồn, thanh ghi đích, hằng số tức thời và thông tin điều khiển luồng chương trình.
- Chuẩn bị các toán hạng đầu vào cho giai đoạn thực thi, bao gồm dữ liệu từ các thanh ghi nguồn và các giá trị hằng số được suy diễn từ lệnh.
- Xác định các yêu cầu điều khiển ở mức kiến trúc, làm cơ sở để các giai đoạn sau thực hiện đúng hành vi của lệnh.
- Hỗ trợ xác định địa chỉ PC kế tiếp đối với các lệnh rẽ nhánh hoặc nhảy, thông qua việc tính toán offset.
- Đảm bảo tính nhất quán của luồng thực thi trong pipeline bằng cách xử lý các tình huống phụ thuộc dữ liệu có thể ảnh hưởng đến tính đúng đắn của kết quả.

Thiết kế các khối phần cứng trong Instruction Decode sau:

4.3.1 Control Unit

Khối Control Unit chịu trách nhiệm sinh toàn bộ các tín hiệu điều khiển của hệ thống dựa trên hai trường `opcode` và `funct` của lệnh. Các tín hiệu này điều khiển hành vi của ALU, Register File, bộ nhớ dữ liệu và luồng điều khiển chương trình (nhánh và nhảy).

Hướng	Tên tín hiệu	Độ rộng	Mô tả chức năng
Input	opcode	4 bit	Mã lệnh chính, xác định nhóm lệnh và hành vi tổng quát của datapath.
	funct	3 bit	Trường chức năng, dùng để phân biệt các lệnh con trong cùng một nhóm `opcode`, đặc biệt đối với các lệnh dạng R-type.
Output	PCSrc	2 bit	Tín hiệu điều khiển nguồn cập nhật Program Counter (PC). `00` – giá trị PC+2 `01` – địa chỉ nhánh (branch_target) `11` – địa chỉ nhảnh của lệnh `jump`
	regDst	1	Chọn thanh ghi đích trong Register File: `0` – trường `rt`, `1` – trường `rd`.
	ALUSrc	1	Chọn toán hạng thứ hai của ALU: `0` – dữ liệu từ Register File, `1` – giá trị tức thời (immediate).
	memWrite	1	Cho phép ghi dữ liệu vào bộ nhớ dữ liệu (Data Memory).
	memRead	1	Cho phép đọc dữ liệu từ bộ nhớ dữ liệu.
	regWrite	1	Cho phép ghi kết quả vào Register File.
	memToReg	1	Chọn nguồn dữ liệu ghi về Register File: `0` – kết quả từ ALU, `1` – dữ liệu đọc từ bộ nhớ.

	r_spec_regs	2	Chọn thanh ghi đặc biệt làm nguồn đọc dữ liệu
			`00` – đọc dữ liệu từ thanh ghi đặc biệt `$ZERO`
			`01` – đọc dữ liệu từ thanh ghi đặc biệt `$RA`
			`10` – đoc dữ liệu từ thanh ghi đặc biệt `$AT`
			`11` – đọc dữ liệu từ thanh ghi đa dụng `$rs`

	w_spec_regs	2	Chọn thanh ghi đặc biệt đích để ghi dữ liệu đối với các lệnh *move to special registers*.
			`00` – ghi dữ liệu vào thanh ghi đặc biệt `$RA`
			`01` – ghi dữ liệu vào thanh ghi đặc biệt `$AT`
			`10` – ghi dữ liệu vào thanh ghi đa dụng

	ALU_cnt	6	Tín hiệu điều khiển ALU, xác định phép toán cụ thể (cộng, trừ, nhân chia, so sánh, v.v)
	is_jump	1	Chỉ thị lệnh nhảy không điều kiện, được sử dụng để điều khiển luồng thực thi chương trình.
	halt	1	Tín hiệu dừng hệ thống khi gặp lệnh `HALT`.

Bảng 6: *Giao diện tín hiệu của module* `control_unit`

Tín hiệu điều khiển `ALU_cnt` được sinh ra từ Control Unit dựa trên trường `opcode` và `funct`. Tín hiệu này xác định phép toán cụ thể mà khối ALU sẽ thực hiện. Bảng 7 mô tả chi tiết mã hoá của tín hiệu `ALU_cnt`.

ALU_cnt	**Nhóm lệnh**	**Chức năng**
000000	ALU0	Cộng hai toán hạng không dấu (addu)
000001	ALU0	Trừ hai toán hạng không dấu (subu)
000010	ALU0	Nhân hai toán hạng không dấu (multu)
000011	ALU0	Chia hai toán hạng không dấu (divu)
000100	ALU0	Phép AND logic
000101	ALU0	Phép OR logic
000110	ALU0	Phép NOR logic
000111	ALU0	Phép XOR logic
001000	ALU1	Cộng hai toán hạng có dấu (add)
001001	ALU1	Trừ hai toán hạng có dấu (sub)
001010	ALU1	Nhân hai toán hạng có dấu (mult)
001011	ALU1	Chia hai toán hạng có dấu (div)
001100	ALU1	So sánh bé hơn (slt)
001101	ALU1	So sánh bằng (seq)
001110	ALU1	So sánh bé hơn không dấu (sltu)
010000	Shift	Dịch phải logic (shr)
010001	Shift	Dịch trái logic (shl)
010010	Shift	Xoay phải (ror)
010011	Shift	Xoay trái (rol)
101000	Immediate	Cộng với hằng số tức thời (addi)
101001	Immediate	So sánh bé hơn với hằng số tức thời (slti)
101010	Branch	So sánh khác nhau, sinh tín hiệu zero (bneq)
101011	Branch	So sánh lớn hơn 0, sinh tín hiệu zero (bgtz)
101100	Memory	Tính địa chỉ cho lệnh LH
101101	Memory	Tính địa chỉ cho lệnh SH
011000	Special Reg	Trả về operand 1 (mfz, mfra, mfat)
011001	Special Reg	Trả về giá trị PC (mfpc)
011100	Special Reg	Trả về giá trị thanh ghi HI (mfhi)
011101	Special Reg	Trả về giá trị thanh ghi LO (mflo)
111111	NOP	ALU ở trạng thái trung tính, không tạo ra kết quả hợp lệ

Bảng 7: *Đặc tả tín hiệu điều khiển `ALU_cnt`*

Hiện thực logic chính:

```
always @(*) begin
        regDst = 1'b0;
        ALUSrc = 1'b0;
        memWrite = 1'b0;
        memRead = 1'b0;
        regWrite = 1'b0;
        memToReg = 1'b0;
        PCSrc = 2'b00;
        r_spec_regs = 2'b11;
        w_spec_regs = 2'b10;
        halt = 1'b0;
        ALU_cnt = 6'b111111;
        is_jump = 1'b0;

        case (opcode)
        4'b0000:
        begin
            regDst = 1'b1;
            ALUSrc = 1'b0;
            memWrite = 1'b0;
            memRead = 1'b0;
            regWrite = (funct == 3'b010 || funct == 3'b011)? 1'b0 : 1'
                b1;
            memToReg = 1'b0;
            PCSrc = 2'b00;
            r_spec_regs = 2'b11;
            w_spec_regs = 2'b10;
            is_jump = 1'b0;
            halt = 1'b0;
            case (funct)
                3'b000: ALU_cnt = 6'b000000; //addu
                3'b001: ALU_cnt = 6'b000001; //subu
                3'b010: ALU_cnt = 6'b000010; //multu
```

```
                3'b011: ALU_cnt = 6'b000011; //divu
                3'b100: ALU_cnt = 6'b000100; //and
                3'b101: ALU_cnt = 6'b000101; //or
                3'b110: ALU_cnt = 6'b000110; //nor
                3'b111: ALU_cnt = 6'b000111; //xor
                default:ALU_cnt = 6'b111111;
            endcase
        end
        4'b0001:
        begin
            regDst = 1'b1;
            ALUSrc = 1'b0;
            memWrite = 1'b0;
            memRead = 1'b0;
            regWrite = (funct == 3'b111 || funct == 3'b010 || funct ==
               3'b011)? 1'b0 : 1'b1;
            memToReg = 1'b0;
            PCSrc = (funct == 3'b111) ? 2'b01 : 1'b00;
            r_spec_regs = 2'b11;
            w_spec_regs = 2'b10;
            halt = 1'b0;
            case (funct)
                3'b000:begin
                    ALU_cnt = 6'b001000; // add
                    is_jump = 1'b0;
                    end
                3'b001:begin
                    ALU_cnt = 6'b001001; // sub
                    is_jump = 1'b0;
                    end
                3'b010:begin
                    ALU_cnt = 6'b001010; // mult
                    is_jump = 1'b0;
```

```
                    end
                3'b011: begin
                    ALU_cnt = 6'b001011; // div
                    is_jump = 1'b0;
                    end
                3'b100: begin
                    ALU_cnt = 6'b001100; // slt
                    is_jump = 1'b0;
                    end
                3'b101:begin
                    ALU_cnt = 6'b001101; // seq
                    is_jump = 1'b0;
                    end
                3'b110: begin
                    ALU_cnt = 6'b001110; // sltu
                    is_jump = 1'b0;
                    end
                3'b111: begin
                    ALU_cnt = 6'b001111; // jr
                    is_jump = 1'b0;
                end
                default: begin
                    ALU_cnt = 6'b111111;
                    is_jump = 1'b0;
                    end
            endcase
        end
        4'b0010:
        begin
            regDst = 1'b1;
            ALUSrc = 1'b0;
            memWrite = 1'b0;
            memRead = 1'b0;
```

```
            regWrite = (funct == 3'b000 || funct == 3'b001 || funct ==
                3'b010 || funct == 3'b011)? 1'b1 : 1'b0;
            memToReg = 1'b0;
            PCSrc = 2'b00;
            r_spec_regs = 2'b11;
            w_spec_regs = 2'b10;
            halt = 1'b0;
            is_jump = 1'b0;
            case (funct)
                3'b000: ALU_cnt = 6'b010000; // shr
                3'b001: ALU_cnt = 6'b010001; // shl
                3'b010: ALU_cnt = 6'b010010; // ror
                3'b011: ALU_cnt = 6'b010011; // rol
                default: ALU_cnt = 6'b111111;
            endcase
        end
        4'b0011: //addi
        begin
            regDst = 1'b0;
            ALUSrc = 1'b1;
            memWrite = 1'b0;
            memRead = 1'b0;
            regWrite = 1'b1;
            memToReg = 1'b0;
            PCSrc = 2'b00;
            r_spec_regs = 2'b11;
            w_spec_regs = 2'b10;
            halt = 1'b0;
            ALU_cnt = 6'b101000;
            is_jump = 1'b0;
        end
        4'b0100: //slti $rt, $rs, i
        begin
```

```
            regDst = 1'b0;
            ALUSrc = 1'b1;
            memWrite = 1'b0;
            memRead = 1'b0;
            regWrite = 1'b1;
            memToReg = 1'b0;
            PCSrc = 2'b00;
            r_spec_regs = 2'b11;
            w_spec_regs = 2'b10;
            halt = 1'b0;
            ALU_cnt = 6'b101001;
            is_jump = 1'b0;
        end
        4'b0101: //bneq
        begin
            regDst = 1'b0; //regDst = x
            ALUSrc = 1'b0;
            memWrite = 1'b0;
            memRead = 1'b0;
            regWrite = 1'b0;
            memToReg = 1'b0;
            PCSrc = 2'b10;
            r_spec_regs = 2'b11;
            w_spec_regs = 2'b10;
            halt = 1'b0;
            ALU_cnt = 6'b101010;
            is_jump = 1'b0;
        end
        4'b0110: //bgtz $rs, i [$rs] > 0?
        begin
            regDst = 1'b0;
            ALUSrc = 1'b1; // so sanh voi $zero
            memWrite = 1'b0;
```

```
            memRead = 1'b0;
            regWrite = 1'b0;
            memToReg = 1'b0; // = x
            PCSrc = 2'b10;
            r_spec_regs = 2'b11;
            w_spec_regs = 2'b10;
            halt = 1'b0;
            ALU_cnt = 6'b101011;
            is_jump = 1'b0;
        end
        4'b0111: // j i
        begin
            regDst = 1'b0;
            ALUSrc = 1'b0; // x
            memWrite = 1'b0;
            memRead = 1'b0;
            regWrite = 1'b0;
            memToReg = 1'b0; //x
            PCSrc = 2'b11;
            r_spec_regs = 2'b11;
            w_spec_regs = 2'b10;
            halt = 1'b0;
            ALU_cnt = 6'b101110;
            is_jump = 1'b1;
        end
        4'b1000:
        begin //lh $rt, i($rs)
            regDst = 1'b0;
            ALUSrc = 1'b1;
            memWrite = 1'b0;
            memRead = 1'b1;
            regWrite = 1'b1;
            memToReg = 1'b1;
```

```
            PCSrc = 2'b00;
            r_spec_regs = 2'b11;
            w_spec_regs = 2'b10;
            halt = 1'b0;
            ALU_cnt = 6'b101100;
            is_jump = 1'b0;
        end
        4'b1001:
        begin
            regDst = 1'b0; // x
            ALUSrc = 1'b1;
            memWrite = 1'b1;
            memRead = 1'b0;
            regWrite = 1'b0;
            memToReg = 1'b0;
            PCSrc = 2'b00;
            r_spec_regs = 2'b11;
            w_spec_regs = 2'b10;
            halt = 1'b0;
            ALU_cnt = 6'b101101;
            is_jump = 1'b0;
        end
        4'b1010: // MFSR Move from special registers kieu R
        begin
            regDst = 1'b1; //rd
            ALUSrc = 1'b0;
            memWrite = 1'b0;
            memRead = 1'b0;
            regWrite = 1'b1;
            memToReg = 1'b0;
            PCSrc = 1'b00;
            halt = 1'b0;
            is_jump = 1'b0;
```

```
            case (funct)
                3'b000: begin
                    r_spec_regs = 2'b00;
                    ALU_cnt = 6'b011000; // mfz
                end
                3'b001: begin
                    r_spec_regs = 2'b11;
                    ALU_cnt = 6'b011001; // mfpc
                end
                3'b010: begin
                    r_spec_regs = 2'b01;
                    ALU_cnt = 6'b011010; // mfra
                end
                3'b011: begin
                    r_spec_regs = 2'b10;
                    ALU_cnt = 6'b011011; // mfat
                end
                3'b100: begin
                    r_spec_regs = 2'b11;
                    ALU_cnt = 6'b011100; // mfhi
                end
                3'b101: begin
                    r_spec_regs = 2'b11;
                    ALU_cnt = 6'b011101; // mflo
                end
                default: begin
                    r_spec_regs = 2'b11;
                    ALU_cnt = 6'b111111;
                end
            endcase
            w_spec_regs = 2'b10;
        end
        4'b1011: // MTSR Move to
```

```
        begin
            regDst = 1'b1; //rd
            ALUSrc = 1'b0;
            memWrite = 1'b0;
            regWrite = (funct == 3'b010 || funct == 3'b011)? 1'b1 : 1'
                b0;
            memToReg = 1'b0;
            PCSrc = 2'b00;
            halt = 1'b0;
            memRead = 1'b0;
            is_jump = 1'b0;
            case (funct)
                3'b010: begin
                    w_spec_regs = 2'b00;
                    ALU_cnt = 6'b100010; // mtra
                end
                3'b011: begin
                    w_spec_regs = 2'b01;
                    ALU_cnt = 6'b100011; // mtat
                end
                3'b100: begin
                    w_spec_regs = 2'b10;
                    hi_write = 1'b1;
                    ALU_cnt = 6'b100100; // mthi
                end
                3'b101: begin
                    w_spec_regs = 2'b10;
                    lo_write = 1'b1;
                    ALU_cnt = 6'b100101; // mtlo
                end
                default: begin
                    w_spec_regs = 2'b10;
                    ALU_cnt = 6'b111111;
```

```
                end
            endcase
            r_spec_regs = 2'b11;
        end
        4'b1111: //HLT signal = 0
        begin
            regDst = 1'b0;
            ALUSrc = 1'b0;
            memWrite = 1'b0;
            regWrite = 1'b0;
            memToReg = 1'b0;
            PCSrc = 2'b00;
            memRead = 1'b0;
            w_spec_regs = 2'b10;
            r_spec_regs = 2'b11;
            halt = 1'b1;
            ALU_cnt = 6'b101111;
        end
        default: ;
    endcase
end
```

4.3.2 Hazard Detection Unit

Khối này có nhiệm vụ phát hiện hazard dữ liệu kiểu *load–use* bằng cách kiểm tra sự phụ thuộc giữa các thanh ghi nguồn của lệnh đang ở ID stage và thanh ghi đích của lệnh ở EX stage. Khi phát hiện xung đột dữ liệu:

- Pipeline được tạm dừng (stall) trong một chu kỳ clock.
- Các tín hiệu điều khiển được vô hiệu hóa nhằm chèn một lệnh *NOP* vào pipeline.

Hướng	Tên tín hiệu	Độ rộng	Mô tả
Input	IF_ID_RegisterRs	3	Thanh ghi nguồn `rs` của lệnh đang ở thanh ghi IF/ID.
	IF_ID_RegisterRt	3	Thanh ghi nguồn `rt` của lệnh đang ở thanh ghi IF/ID.
	ID_EX_RegisterRt	3	Thanh ghi đích `rt` của lệnh load đang ở thanh ghi ID/EX.
	ID_EX_MemRead	1	Chỉ thị lệnh ở thanh ghi ID/EX có thực hiện đọc bộ nhớ hay không.
Output	Write	1	Điều khiển cho phép cập nhật PC và thanh ghi IF/ID. Giá trị `0` tương ứng với việc chèn stall.
	HazardMuxControl	1	Tín hiệu điều khiển MUX để vô hiệu hóa các tín hiệu điều khiển khi xảy ra hazard.

Bảng 8: *Giao diện tín hiệu của module* ***hazard_detection_unit***

Hiện thực logic chính:

```
always @(*) begin
    if (ID_EX_MemRead &&
        ((ID_EX_RegisterRt == IF_ID_RegisterRs)
         || (ID_EX_RegisterRt == IF_ID_RegisterRt))) begin
        Write = 1'b0;
        HazardMuxControl = 1'b1;
    end
    else begin
        Write = 1'b1;
        HazardMuxControl = 1'b0;
    end
end
```

4.3.3 MUX Hazard

Khối *MUX Hazard* được sử dụng để vô hiệu hóa các tín hiệu điều khiển trong trường hợp xảy ra stall do hazard. Khi tín hiệu `stall` được kích hoạt từ khối Hazard Detection Unit, MUX Hazard sẽ chọn các giá trị điều khiển trung tính (NOP) thay vì các tín hiệu điều khiển hợp lệ từ Control Unit.

Bảng tín hiệu của MUX Hazard

Hướng	Tên tín hiệu	Độ rộng	Mô tả chức năng
Input	HazardMuxControl	1	Tín hiệu điều khiển MUX Hazard, kích hoạt khi xảy ra hazard để vô hiệu hóa các tín hiệu điều khiển.
	ctrl_in	17	Bus các tín hiệu điều khiển từ Control Unit.
Output	ctrl_out	17	Bus tín hiệu điều khiển sau khi được chọn lọc, đưa đến các stage sau của pipeline.

Bảng 9: *Giao diện tín hiệu của module `MUX_Hazard`*

Trong đó, bus `ctrl_in` bao gồm các tín hiệu điều khiển như `regWrite`, `memWrite`, `memRead`, `memToReg`, `ALUSrc`, `ALU_cnt`, `PCSrc`, `halt`, `regDst`, `r_spec_regs` và `w_spec_regs`. Bus `ctrl_out` là tập tín hiệu điều khiển đầu ra tương ứng, được tạo ra sau khi khối phát hiện hazard kiểm tra và xử lý tình huống *load-use hazard*, nhằm đảm bảo luồng thực thi trong pipeline diễn ra đúng đắn.

Nguyên lý họa động: Khi `HazardMuxControl = 0`, MUX Hazard truyền trực tiếp các tín hiệu điều khiển từ Control Unit. Khi `HazardMuxControl = 1`, MUX Hazard ép tất cả các tín hiệu điều khiển về mức không kích hoạt, tương đương với việc chèn một lệnh NOP vào pipeline.

Hiện thực logic chính:

```
    always @(*) begin
        if (HazardMuxControl) begin
            regDst_out = 1'b0;
            ALU_cnt_out = 6'b111111;
            ALUSrc_out = 1'b0;
            memRead_out = 1'b0;
            memWrite_out = 1'b0;
            regWrite_out = 1'b0;
            memToReg_out = 1'b0;
            PCSrc_out = 2'b00;
            r_spec_regs_out = 2'b11;
            w_spec_regs_out = 2'b10;
            halt_out = 1'b0;
            is_jump_out = 1'b0;
        end
        else begin
            regDst_out = regDst;
```

```
            ALU_cnt_out = ALU_cnt;
            ALUSrc_out = ALUSrc;
            memRead_out = memRead;
            memWrite_out = memWrite;
            regWrite_out = regWrite;
            memToReg_out = memToReg;
            PCSrc_out = PCSrc;
            r_spec_regs_out = r_spec_regs;
            w_spec_regs_out = w_spec_regs;
            halt_out = halt;
            is_jump_out = is_jump;
        end
    end

    assign is_jump = controlunit_is_jump && !HazardControl; // trong
        module id_stage
```

4.3.4 Register File

Register File lưu trữ các thanh ghi phục vụ cho quá trình thực thi lệnh, bao gồm 8 thanh ghi đa dụng `$0-$7` và 3 thanh ghi đặc biệt `$ZERO`, `$AT`, `$RA`. Hai thanh ghi đặc biệt `$HI` và `$LO` được cài đặt và quản lý riêng trong khối ALU.

Nguyên lý hoạt động: Register File hoạt động như một khối lưu trữ trung tâm, cung cấp dữ liệu toán hạng cho các giai đoạn thực thi và nhận kết quả ghi trả về ở giai đoạn Write Back. Khối này hỗ trợ cơ chế đọc không đồng bộ và ghi đồng bộ.

- Trong mỗi chu kỳ, Register File đồng thời nhận chỉ số hai thanh ghi nguồn `rs` và `rt`. Dữ liệu tương ứng được truy xuất và xuất ra các cổng `rd1` và `rd2`. Quá trình đọc diễn ra không phụ thuộc vào xung clock, cho phép dữ liệu được sử dụng ngay trong cùng chu kỳ tại các giai đoạn tiếp theo của pipeline.
- Đối với các lệnh truy cập thanh ghi đặc biệt (*move from special registers*), nguồn dữ liệu tại cổng `rd1` không được lấy từ tập thanh ghi đa dụng mà được chọn từ các thanh ghi đặc biệt dựa trên tín hiệu điều khiển `r_spec_regs`.
- Register File nhận tín hiệu reset bất đồng bộ `rst`. Khi `rst` được kích hoạt, toàn bộ các thanh ghi

được đưa về giá trị khởi tạo xác định, độc lập với xung `clk`.

- Việc ghi dữ liệu vào Register File được thực hiện đồng bộ với cạnh lên của xung `clk`. Khi tín hiệu `regWrite` được kích hoạt, giá trị tại ngõ vào `write_data` sẽ được ghi vào thanh ghi đích được chỉ định bởi `write_reg`. Nếu `regWrite` không được kích hoạt, nội dung các thanh ghi được giữ nguyên.
- Đối với các lệnh ghi vào thanh ghi đặc biệt (*move to special registers*), việc ghi không diễn ra trong Register File thông thường mà được điều khiển thông qua tín hiệu `w_spec_regs`.

Hướng	**Tên tín hiệu**	**Độ rộng**	**Mô tả**
Input	clk	1	Xung clock đồng bộ cho quá trình ghi dữ liệu vào Register File.
	rst	1	Tín hiệu reset, đưa các thanh ghi về trạng thái ban đầu.
Input (từ WB stage)	reg_write	1	Cho phép ghi dữ liệu vào Register File.
	write_reg	3	Địa chỉ thanh ghi đích để ghi dữ liệu (0–7).
	write_data	16	Dữ liệu được ghi vào thanh ghi đích.
	w_spec_regs	2	Chọn thanh ghi đặc biệt đích để ghi dữ liệu trong các lệnh *move to special registers*.
Input (từ thanh ghi IF/ID)	rs	3	Địa chỉ thanh ghi nguồn thứ nhất .
	rt	3	Địa chỉ thanh ghi nguồn thứ hai.
Input (từ Control Unit)	r_spec_regs	2	Chọn thanh ghi đặc biệt làm nguồn đọc dữ liệu.
Output	rd1	16	Dữ liệu đọc từ thanh ghi nguồn `rs`. Dữ liệu đọc từ thanh ghi đặc biệt, áp dụng cho nhóm lệnh *move from special registers*.
	rd2	16	Dữ liệu đọc từ thanh ghi nguồn `rt`.

Bảng 10: *Giao diện tín hiệu của module `registers_file`*

Hiện thực logic chính:

```
reg [15:0] regs [0:10];  // 11 registers total
reg [3:0] actual_write_reg;  // 4-bit to support up to reg 12

integer i;

// Determine actual write register based on w_spec_regs
always @(*) begin
    case (w_spec_regs)
```

```
            2'b00: actual_write_reg = 4'd10; //ra
            2'b01: actual_write_reg = 4'd9; //at
            default: actual_write_reg = {1'b0, write_reg};
        endcase
    end

    integer i;
    // WRITE: Sequential logic (on clock edge)
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            // Initialize all registers to 0
            for (i = 0; i <= 10; i = i + 1) begin
                regs[i] <= 16'd0;
            end
        end
        else begin
            // Write to register (but $0 and $8=$zero are hardwired to 0)
            if (reg_write && actual_write_reg != 4'd8) begin
                regs[actual_write_reg] <= write_data;
            end
        end
    end

    // READ: Combinatorial logic for rd1 (with special register support)
    reg [15:0] rd1_data;
    always @(*) begin
        case (r_spec_regs)
            2'b00: rd1_data = (actual_write_reg == 4'd8 && reg_write)? write_data : regs[4'd8];      // $zero
            2'b10: rd1_data = (actual_write_reg == 4'd9 && reg_write)? write_data : regs[4'd9];        // $at
```

```
            2'b01: rd1_data = (actual_write_reg == 4'd10 && reg_write)?
                write_data : regs[4'd10];        // $ra
            default: begin
               rd1_data = (actual_write_reg == rs && reg_write) ?
                  write_data : regs[{1'b0, rs}];
            end
        endcase
    end

    assign rd1 = rd1_data;
    assign rd2 = (actual_write_reg == rt && reg_write) ? write_data :
       regs[{1'b0, rt}];
```

4.3.5 Sign Extend

Khối Sign Extend thực hiện mở rộng dấu trường `immediate` từ 6 bits lên 16 bits. Giá trị mở rộng này được sử dụng trong:

- Các phép toán ALU của các lệnh dạng I-type.
- Việc tính toán địa chỉ nhánh (branch target) trong các lệnh rẽ nhánh.

Hướng	Tên tín hiệu	Độ rộng	Mô tả
Input	immediate	6	Giá trị tức thời trích xuất từ trường `immediate` của lệnh, được dùng làm toán hạng cho các phép toán ALU.
Output	sign_extend	16	Giá trị tức thời sau khi được mở rộng dấu từ 6 bits lên 16 bits, bảo toàn giá trị số có dấu ban đầu.

Bảng 11: *Giao diện tín hiệu của module `Sign Extend`*

Hiện thực logic chính:

```
    always @(*) begin
        sign_extend = {{10{immediate[5]}}, immediate};
    end
```

4.3.6 Shift left 1

Hai khối *shift left 1* được sử dụng để hỗ trợ việc tính toán địa chỉ rẽ nhánh (branch target) và địa chỉ nhảy của lệnh `jump`.

Nguyên lý hoạt động được mô tả như sau:

- **Địa chỉ nhảy (jump address):** Trường địa chỉ trong lệnh `jump` được dịch trái 1 bit, sau đó ghép với các bit cao của PC hiện tại để tạo thành địa chỉ nhảy đầy đủ.
- **Địa chỉ rẽ nhánh (branch target):** Giá trị hằng số tức thời sau khi được mở rộng dấu sẽ được dịch trái 1 bit và cộng với PC hiện tại để xác định địa chỉ đích của lệnh rẽ nhánh.

Hiện thực logic chính:

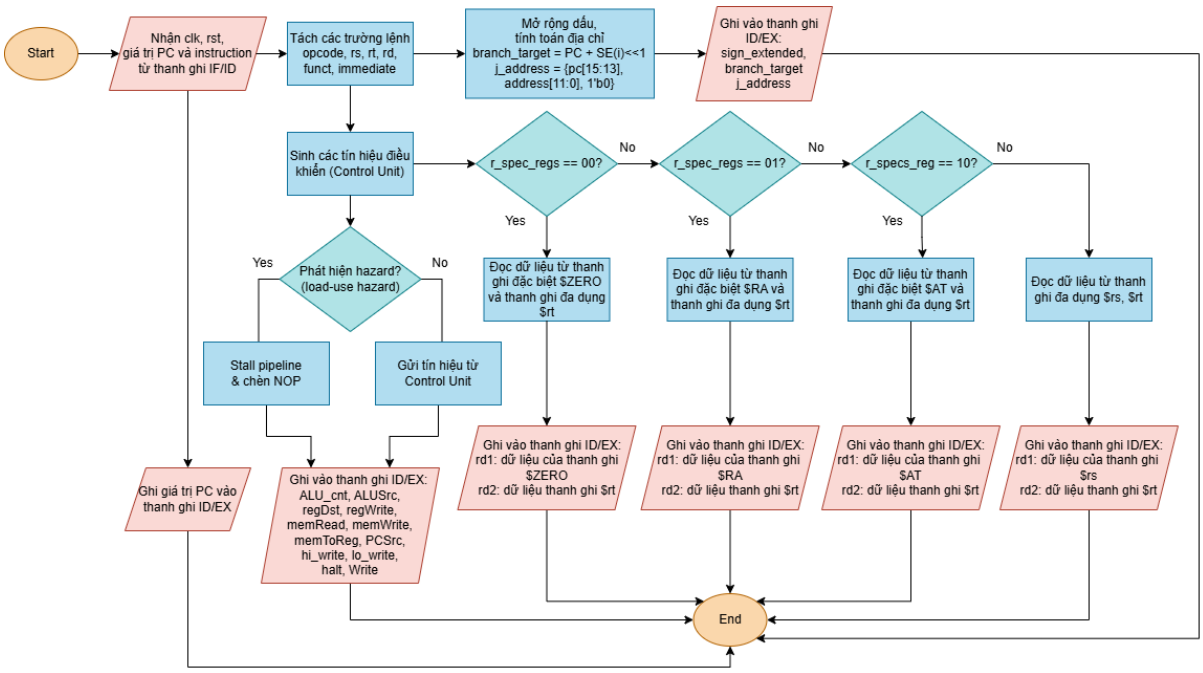
```
assign j_address = {pc[15:13], address[11:0], 1'b0};
assign branch_target = pc + (extended_int << 1);
```

4.3.7 Bảng tín hiệu ID Stage (Interface)

Hướng	Tên tín hiệu	Độ rộng	Mô tả chức năng
Input	clk	1	Xung nhịp hệ thống
	rst	1	Tín hiệu reset hệ thống
	pc	16	Giá trị Program Counter từ IF stage
	inst	16	Lệnh được lấy từ bộ nhớ lệnh
	regWrite_wb	1	Cho phép ghi Register File từ WB stage
	write_reg	3	Chỉ số thanh ghi đích ghi về từ WB
	write_data	16	Dữ liệu ghi về Register File từ WB stage
	w_spec_regs_wb	2	Điều khiển ghi thanh ghi đặc biệt từ WB stage
Input (Hazard)	ex_memRead	1	Cho biết lệnh ở EX stage là load
	ex_rt	3	Thanh ghi đích của lệnh ở EX stage
Output (Data)	rd1	16	Dữ liệu đọc từ thanh ghi rs. Dữ liệu đọc từ thanh ghi đặc biệt, áp dụng cho nhóm lệnh *move from special registers.*
	rd2	16	Dữ liệu đọc từ thanh ghi rt
	id_rs	3	Chỉ số thanh ghi rs
	id_rt	3	Chỉ số thanh ghi rt
	id_rd	3	Chỉ số thanh ghi rd
	extended_int	16	Immediate đã được mở rộng dấu
	pc_out	16	Giá trị PC truyền sang stage kế tiếp
Output (Control Flow)	branch_target	16	Địa chỉ nhánh được tính toán
	j_address	16	Địa chỉ nhảy (jump target)
	is_jump	1	Tín hiệu xác định lệnh nhảy hợp lệ
Output (Control)	ALU_cnt	6	Tín hiệu điều khiển phép toán ALU
	ALUSrc	1	Chọn nguồn toán hạng thứ hai cho ALU
	regDst	1	Chọn thanh ghi đích
	regWrite	1	Cho phép ghi thanh ghi
	memRead	1	Cho phép đọc bộ nhớ
	memWrite	1	Cho phép ghi bộ nhớ
	memToReg	1	Chọn dữ liệu ghi về thanh ghi
	PCSrc	2	Điều khiển cập nhật Program Counter
	halt	1	Dừng hoạt động pipeline
Output (Hazard)	Write	1	Cho phép cập nhật IF/ID register

Bảng 12: *Bảng tín hiệu của Instruction Decode (ID) Stage*

4.3.8 Luồng xử lý dữ liệu



Hình 4.3: *Luồng xử lý dự liệu giai đoạn ID*

4.4 Thiết kế và hiện thực Execute

Giai đoạn Execute (EX) là trung tâm xử lý số học và logic của CPU, thực hiện các phép toán trên dữ liệu từ thanh ghi hoặc giá trị immediate. Giai đoạn này bao gồm ALU (Arithmetic Logic Unit), các multiplexer để chọn toán hạng, và cơ chế forwarding để giải quyết data hazard. EX cũng tính toán địa chỉ cho các lệnh branch và xác định điều kiện rẽ nhánh. Với các module thành phần:

4.4.1 Multiplexer Alrithmetic Logic Unit (MUX_ALU)

Module MUX_ALU là multiplexer đơn giản quyết định toán hạng thứ hai của ALU: dữ liệu từ thanh ghi (rs2) hoặc giá trị immediate đã được mở rộng dấu.

Tín hiệu	Độ rộng	Hướng	Chức năng
ALU_Src	1	Input	Tín hiệu điều khiển từ Control Unit.
rs2	16	Input	Dữ liệu thanh ghi nguồn 2 sau khi forwarding.
sign_extend	16	Input	Giá trị immediate đã mở rộng dấu từ giai đoạn ID.
out	16	Output	Toán hạng thứ hai cho ALU.

Bảng 13: *Các tín hiệu trong MUX_ALU*

Nguyên lý hoạt động: Khi ALUSrc = 1 (các lệnh I-type như ADDI, SLTI, LW, SW), module chọn sign_extend. Khi ALUSrc = 0 (như các lệnh R-type), module chọn rs2 từ thanh ghi. Đây là phép chọn tổ hợp không có trễ.

Hiện thực logic chính:

```
assign out = ALUSrc ? sign_extend : rs2;
```

4.4.2 Multiplexer 3 to 1 (MUX_3_to_1)

Module MUX_3_to_1 là multiplexer 3 đầu vào được sử dụng để thực hiện data forwarding, cho phép chuyển tiếp dữ liệu từ các giai đoạn MEM hoặc WB về EX mà không cần chờ ghi vào Register File. Module này được sử dụng hai lần trong ALU_System: một cho toán hạng rs1 (ForwardA) và một cho toán hạng rs2 (ForwardB).

Tín hiệu	Độ rộng	Hướng	Chức năng
in0	16	Input	Dữ liệu từ pipeline register ID/EX (không forward).
in1	16	Input	Dữ liệu từ giai đoạn WB (forward từ WB).
in2	16	Input	Dữ liệu từ giai đoạn MEM (forward từ MEM).
select	2	Input	Tín hiệu chọn từ Fowarding Unit.
out	16	Output	Dữ liệu đã được chọn.

Bảng 14: *Các tín hiệu trong MUX_3_to_1*

Nguyên lý hoạt động: Dựa trên giá trị select:

- 2'b00: Không forward, sử dụng dữ liệu từ ID/EX.
- 2'b01: Forward từ WB (MEM_WB_write_data).
- 2'b10: Forward từ MEM (EX_MEM_alu_result).

Module này được sử dụng hai lần trong EX: một cho toán hạng rs1 (ForwardA) và một cho toán hạng rs2 (ForwardB).

Hiện thực logic chính:

```
always @(*) begin
    case (select)
        2'b00: out = in0;
        2'b01: out = in1;
        2'b10: out = in2;
```

```
            default: out = in0;
        endcase
    end
```

4.4.3 Alrithmetic Logic Unit (ALU)

ALU là đơn vị thực hiện tất cả các phép toán số học, logic, dịch bit và so sánh. Đây là module phức tạp nhất trong giai đoạn EX với hơn 30 phép toán khác nhau. Khác với các kiến trúc RISC hoặc MIPS tiêu chuẩn thường sử dụng tín hiệu *ALUOp* kết hợp với trường *function* để sinh ra tín hiệu điều khiển ALU, trong thiết kế của nhóm không tồn tại tín hiệu *ALUOp*. Thay vào đó, khối điều khiển tại giai đoạn *ID* thực hiện trực tiếp việc giải mã lệnh và sinh ra tín hiệu *ALUControl*, là tín hiệu được đưa đến ALU để điều khiển phép toán cần thực hiện. Việc loại bỏ tín hiệu *ALUOp* trong thiết kế mang lại cả ưu điểm và hạn chế nhất định. Về ưu điểm, toàn bộ logic xác định phép toán của ALU được xử lý trực tiếp tại Control Unit ở giai đoạn ID. Do đó, ALU chỉ cần nhận một tín hiệu duy nhất là *ALUControl*, không cần phải truyền thêm các trường *opcode* và *function* của lệnh. Cách tiếp cận này giúp giảm số lượng tín hiệu điều khiển, đơn giản hóa datapath và tránh việc phải xử lý nhiều nhóm lệnh có mã opcode khác biệt so với kiến trúc MIPS tiêu chuẩn.

Tuy nhiên, nhược điểm của phương pháp này là khối Control Unit trở nên phức tạp hơn do phải đảm nhiệm toàn bộ việc giải mã lệnh và sinh tín hiệu điều khiển ALU. Bên cạnh đó, thiết kế có xu hướng phụ thuộc chặt vào tập lệnh hiện tại; khi mở rộng hoặc thay đổi Instruction Set, Control Unit cần được chỉnh sửa đáng kể, làm giảm tính linh hoạt và khả năng tái sử dụng của hệ thống so với cách tiếp cận tách riêng *ALUOp* và khối ALU Control trong các kiến trúc truyền thống.

Ngoài ra, theo đặc tả đồ án, hệ thống yêu cầu hỗ trợ các thanh ghi đặc biệt *HI* và *LO*. Trong thiết kế của nhóm, các thanh ghi đa dụng và một số thanh ghi đặc biệt như *PC* hoặc *RA* được tổ chức trong khối Register File. Riêng đối với hai thanh ghi *HI* và *LO*, nhóm lựa chọn tích hợp trực tiếp trong khối ALU thay vì đặt chung trong Register File.

Cách tiếp cận này xuất phát từ việc *HI* và *LO* chỉ được sử dụng trong một số lệnh đặc biệt liên quan đến các phép toán của ALU, chẳng hạn như phép nhân hoặc chia. Việc đặt *HI* và *LO* trong ALU giúp đơn giản hóa Register File, giảm số lượng cổng đọc/ghi cần thiết và làm cho datapath tổng thể gọn hơn. Trong thiết kế của nhóm, việc truy xuất và cập nhật các thanh ghi đặc biệt *HI* và *LO* được thực hiện thông qua các lệnh chuyên biệt như lệnh *MFHI* và *MFLO*, hệ thống sẽ đọc giá trị tương ứng từ các thanh ghi *HI* hoặc *LO* và đưa dữ liệu này về Register File để ghi vào thanh ghi đích. Ngược lại, đối với các lệnh *MTHI* và *MTLO*, giá trị của các thanh ghi *HI* và *LO* được cập nhật trực tiếp tại giai đoạn *EX*

dựa trên dữ liệu đầu vào của ALU.

Tín hiệu	Độ rộng	Hướng	Chức năng
rs1	16	Input	Toán hạng thứ nhất sau forwarding.
out	16	Input	Toán hạng thứ hai từ MUX_ALU.
ALU_cnt	6	Input	Mã điều khiển ALU sinh từ Control Unit.
PC_in	16	Input	Giá trị PC lệnh hiện tại dành cho các lệnh đặc biệt.
result	16	Input	Kết quả phép toán.
zero	1	Input	Tín hiệu zero flag cho các lệnh branch.
mul_result	32	Thanh ghi nội bộ	Kết quả phép nhân 32-bit.
div_hi	16	Thanh ghi nội bộ	Phần cao của phép chia/nhân (HI register).
div_lo	16	Thanh ghi nội bộ	Phần thấp của phép chia/nhân (LO register).

Bảng 15: *Các tín hiệu trong ALU*

Hiện thực logic chính:

```
reg[31:0] mul_result;
reg[15:0] div_hi;
reg[15:0] div_lo;
reg zero_t;

initial begin
    div_hi = 0;
    div_lo = 0;
end

always @(*) begin
    // Default values
    result = 16'b0;
    mul_result = 32'b0;
    case (ALU_cnt)
        // Unsigned arithmetic
        6'b000000: result = $unsigned(rs1) + $unsigned(out);  // addu
        6'b000001: result = $unsigned(rs1) - $unsigned(out);  // subu
        6'b000010: begin  // multu
            mul_result = $unsigned(rs1) * $unsigned(out);
```

```
            result = mul_result[15:0];
            div_hi = mul_result[31:16];
            div_lo = mul_result[15:0];
        end
        6'b000011: begin  // divu
            if (out != 0) begin
                div_hi = $unsigned(rs1) / $unsigned(out);
                div_lo = $unsigned(rs1) % $unsigned(out);
                result = div_lo;
            end
        end

        // Logical operations
        6'b000100: result = rs1 & out;   // and
        6'b000101: result = rs1 | out;   // or
        6'b000110: result = ~(rs1 | out); // nor
        6'b000111: result = rs1 ^ out;   // xor

        // Signed arithmetic
        6'b001000: result = $signed(rs1) + $signed(out);  // add
        6'b001001: result = $signed(rs1) - $signed(out);  // sub
        6'b001010: begin  // mult
            mul_result = $signed(rs1) * $signed(out);
            div_hi = mul_result[31:16];
            div_lo = mul_result[15:0];
            result = mul_result[15:0];
        end
        6'b001011: begin  // div
            if (out != 0) begin
                div_hi = $signed(rs1) % $signed(out);
                div_lo = $signed(rs1) / $signed(out);
                result = div_lo;
            end
```

```
        end
        6'b001100: result = ($signed(rs1) < $signed(out)) ? 16'h0001 : 16'h0000;  // slt
        6'b001101: result = ($signed(rs1) == $signed(out)) ? 16'h0001 : 16'h0000; // seq
        6'b001110: result = ($unsigned(rs1) < $unsigned(out)) ? 16'h0001 : 16'h0000; // sltu
        6'b001111: result = rs1;  // jr

        // Shift operations
        6'b010000: result = out >> rs1[3:0];  // shr
        6'b010001: result = out << rs1[3:0];  // shl
        6'b010010: result = (out >> rs1[3:0]) | (out << (16 - rs1[3:0]));  // ror
        6'b010011: result = (out << rs1[3:0]) | (out >> (16 - rs1[3:0]));       // rol

        // Move from special registers
        6'b011000: result = rs1;  // mfz
        6'b011001: result = PC_in;              // mfpc
        6'b011010: result = rs1;             // mfra
        6'b011011: result = rs1;             // mfat
        6'b011100: result = div_hi;             // mfhi
        6'b011101: result = div_lo;             // mflo

        // I-type instructions
        6'b101000: result = $signed(rs1) + $signed(out);  // addi
        6'b101001: result = ($signed(rs1) < $signed(out)) ? 16'h0001 : 16'h0000;  // slti
        6'b101010: zero_t = (rs1 != out) ? 1'b1 : 1'b0;     // bneq
        6'b101011: zero_t = ($signed(rs1) > $signed(0)) ? 1'b1 : 1'b0;  // bgtz
        6'b101100: result = ((rs1[15:1] + out[15:1]) << 1);  // lh
```

```
        6'b101101: result = ((rs1[15:1] + out[15:1]) << 1);  // sh
        6'b101110: result = {PC_in[15:13], (out[12:0] << 1)};  // ji
        6'b101111: result = 16'b0;  // HALT

         6'b100010: result = out;// mtra
         6'b100011: result = out;// mtat
         6'b100100: div_hi = out;// mthi
         6'b100101: div_lo = out;// mtlo
        default: result = 16'hxxxx;
    endcase
end

assign zero = (ALU_cnt == 6'b101010 || ALU_cnt == 6'b101011)? zero_t :
   1'b0;
```

4.4.4 Forwarding Unit (Forwarding_Unit)

Forwarding_Unit là module quan trọng để giải quyết data hazard trong pipeline, phát hiện khi một lệnh cần dữ liệu từ lệnh trước chưa được ghi vào Register File và chuyển tiếp dữ liệu trực tiếp từ giai đoạn MEM hoặc WB.

Tín hiệu	Độ rộng	Hướng	Chức năng
ID_EX_Rs	3	Input	Thanh ghi nguồn Rs ở EX stage
ID_EX_Rt	3	Input	Thanh ghi nguồn Rt ở EX stage
ID_EX_r_spec_regs	2	Input	Xác định thanh ghi đặc biệt được đọc
EX_MEM_Rd	3	Input	Thanh ghi đích từ EX/MEM stage
EX_MEM_RegWrite	1	Input	Cho phép ghi thanh ghi tại EX/MEM
EX_MEM_w_spec_regs	2	Input	Xác định thanh ghi đặc biệt được ghi
MEM_WB_Rd	3	Input	Thanh ghi đích từ MEM/WB stage
MEM_WB_RegWrite	1	Input	Cho phép ghi thanh ghi tại MEM/WB
MEM_WB_w_spec_regs	2	Input	Xác định thanh ghi đặc biệt được ghi
ForwardA	2	Output	Tín hiệu điều khiển forward cho toán hạng ALU thứ nhất
ForwardB	2	Output	Tín hiệu điều khiển forward cho toán hạng ALU thứ hai

Bảng 16: *Các tín hiệu trong Forwarding Unit*

Nguyên lý hoạt động: Forwarding Unit được sử dụng để giải quyết các *data hazard* trong pipeline bằng

cách chuyển tiếp trực tiếp dữ liệu từ các giai đoạn sau về giai đoạn Execute.

Hoạt động của ForwardA (đối với toán hạng rs1):

Logic forward được kiểm tra theo thứ tự ưu tiên như sau:

- **EX hazard (ưu tiên cao nhất):** Nếu lệnh ở giai đoạn MEM sẽ ghi thanh ghi (`EX_MEM_RegWrite = 1`) và thanh ghi đích trùng với thanh ghi nguồn rs1 của lệnh ở giai đoạn EX (`EX_MEM_Rd == ID_EX_Rs`), khi đó đặt `ForwardA = 2'b10` để forward dữ liệu từ giai đoạn MEM.
- **MEM hazard (ưu tiên thấp hơn):** Nếu lệnh ở giai đoạn WB sẽ ghi thanh ghi và thanh ghi đích trùng với rs1, khi đó đặt `ForwardA = 2'b01` để forward dữ liệu từ giai đoạn WB.
- **Không có hazard:** Trong các trường hợp còn lại, đặt `ForwardA = 2'b00`, dữ liệu được lấy trực tiếp từ thanh ghi.

Hoạt động của ForwardB (đối với toán hạng rs2/rt):

Logic của ForwardB tương tự như ForwardA nhưng phức tạp hơn do cần xử lý cả các thanh ghi thông thường và các thanh ghi đặc biệt. Cơ chế forward được kích hoạt khi thỏa mãn một trong các điều kiện sau:

- Thanh ghi đích của lệnh ở các giai đoạn sau trùng với rt của lệnh hiện tại (`EX_MEM_Rd == ID_EX_Rt`).
- Cả lệnh ghi và lệnh đọc đều truy cập thanh ghi đặc biệt `HI` (`w_spec_regs = 2'b00` và `r_spec_regs = 2'b00`).
- Lệnh ghi vào thanh ghi `LO` (`w_spec_regs = 2'b01`) và lệnh đọc từ thanh ghi `AT` (`r_spec_regs = 2'b10`).

Cơ chế forwarding này đảm bảo các lệnh liên tiếp có phụ thuộc dữ liệu vẫn được thực thi chính xác mà không cần stall pipeline, từ đó cải thiện hiệu năng tổng thể của bộ xử lý.

Hiện thực logic chính:

```
    always @(*) begin

        if (EX_MEM_RegWrite && (EX_MEM_Rd == ID_EX_Rs))begin
            ForwardA = 2'b10;
        end

```

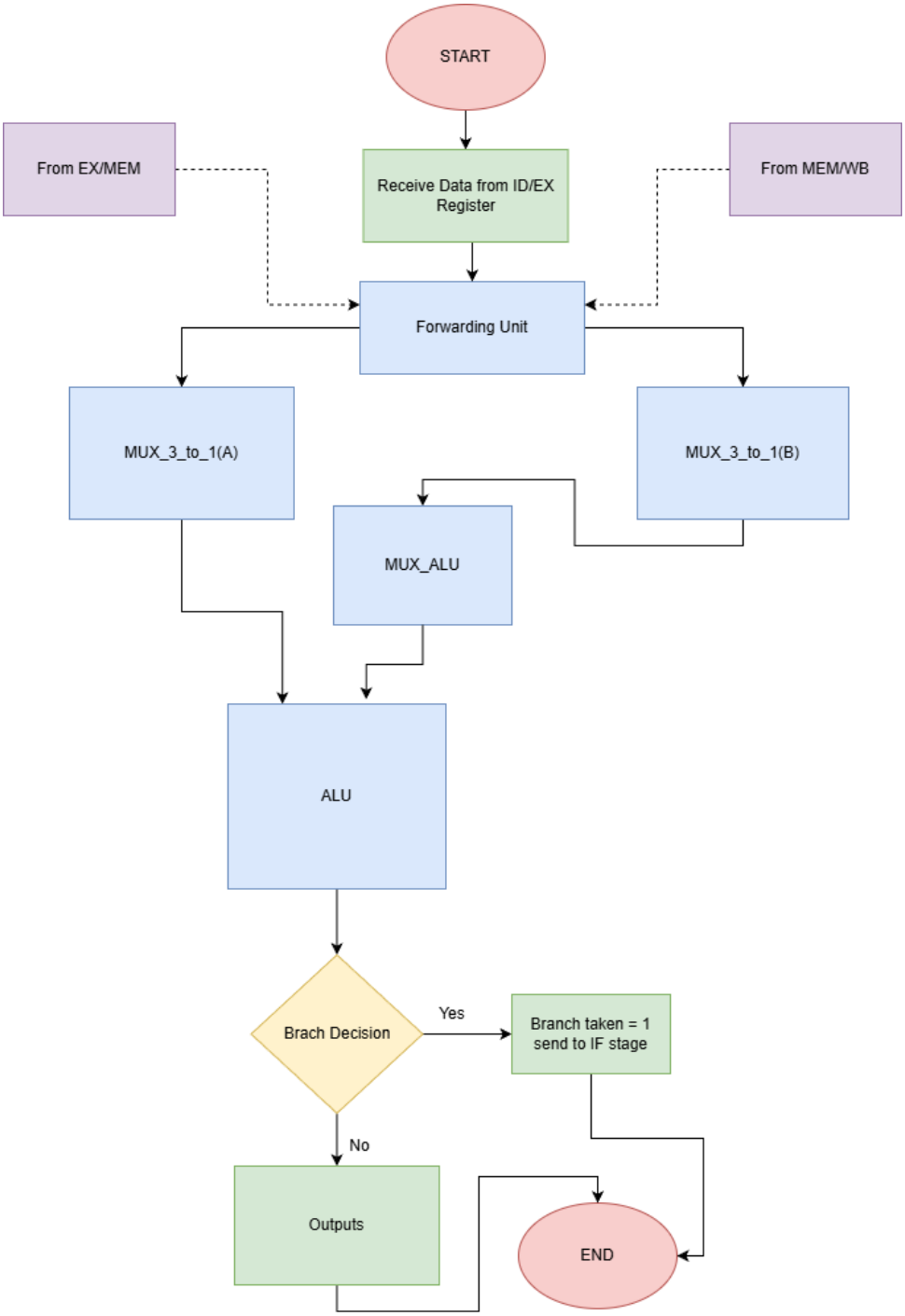
```
        else if (MEM_WB_RegWrite && (MEM_WB_Rd == ID_EX_Rs)) begin
            ForwardA = 2'b01;
        end

        else begin
            ForwardA = 2'b00;
         end
    end

    always @(*) begin
        if (EX_MEM_RegWrite && ((EX_MEM_Rd == ID_EX_Rt) ||
            (EX_MEM_w_spec_regs == 2'b00 && ID_EX_r_spec_regs == 2'b00)
                ||
            (EX_MEM_w_spec_regs == 2'b01 && ID_EX_r_spec_regs == 2'b10)
               )) begin
            ForwardB = 2'b10;
        end

        else if (MEM_WB_RegWrite && ((MEM_WB_Rd == ID_EX_Rt) ||
            (MEM_WB_w_spec_regs == 2'b00 && ID_EX_r_spec_regs == 2'b00)
                ||
            (MEM_WB_w_spec_regs == 2'b01 && ID_EX_r_spec_regs == 2'b10)
               ))begin
            ForwardB = 2'b01;
        end

        else begin
            ForwardB = 2'b00;
        end
    end
```

4.4.5 Bảng tín hiệu EX Stage (Interface)

Tên tín hiệu	Loại	Độ rộng	Mô tả
Clock và Reset			
clk	Input	1-bit	Tín hiệu clock đồng bộ hệ thống
rst	Input	1-bit	Tín hiệu reset hệ thống
Dữ liệu từ ID/EX Pipeline Register			
ID_EX_Rs	Input	3-bit	Địa chỉ thanh ghi nguồn 1
ID_EX_Rt	Input	3-bit	Địa chỉ thanh ghi nguồn 2
ID_EX_rd1	Input	16-bit	Dữ liệu từ thanh ghi Rs
ID_EX_rd2	Input	16-bit	Dữ liệu từ thanh ghi Rt
ID_EX_sign_extend	Input	16-bit	Giá trị immediate đã mở rộng dấu
ID_EX_PC	Input	16-bit	Giá trị Program Counter
ID_EX_r_spec_regs	Input	2-bit	Loại thanh ghi đặc biệt đọc
Control Signals			
ALU_cnt	Input	6-bit	Mã điều khiển ALU (chọn phép toán)
ALUSrc	Input	1-bit	Chọn toán hạng 2: 0=thanh ghi, 1=immediate
PCSrc	Input	2-bit	Nguồn PC (dùng cho branch decision)
Forwarding từ EX/MEM			
EX_MEM_Rd	Input	3-bit	Địa chỉ thanh ghi đích ở MEM
EX_MEM_RegWrite	Input	1-bit	Tín hiệu ghi thanh ghi ở MEM
EX_MEM_alu_result	Input	16-bit	Kết quả ALU từ MEM để forward
EX_MEM_w_spec_regs	Input	2-bit	Loại thanh ghi đặc biệt ghi ở MEM
Forwarding từ MEM/WB			
MEM_WB_Rd	Input	3-bit	Địa chỉ thanh ghi đích ở WB
MEM_WB_RegWrite	Input	1-bit	Tín hiệu ghi thanh ghi ở WB
MEM_WB_write_data	Input	16-bit	Dữ liệu từ WB để forward
MEM_WB_w_spec_regs	Input	2-bit	Loại thanh ghi đặc biệt ghi ở WB
Outputs			
alu_result	Output	16-bit	Kết quả phép toán từ ALU
forward_rd2	Output	16-bit	Dữ liệu Rs2 sau forwarding (cho SW)
forward_rd1	Output	16-bit	Dữ liệu Rs1 sau forwarding
zero	Output	1-bit	Zero flag từ ALU (cho branch)
branch_taken	Output	1-bit	Tín hiệu branch được thực hiện

Bảng 17: *Bảng tín hiệu EX Stage*

4.4.6 Luồng xử lý dữ liệu



Hình 4.4: *Mô tả luồng hoạt động của EX*

4.5 Thiết kế và hiện thực Memory Access

Nhiệm vụ chính của Memory Access (MEM) là:

- **Truy xuất dữ liệu:** Đọc dữ liệu từ bộ nhớ (đối với lệnh Load) hoặc ghi dữ liệu vào bộ nhớ (đối với lệnh Store).
- **Chuyển tiếp kết quả:** Đối với các lệnh không truy cập bộ nhớ, MEM đóng vai trò trung gian chuyển kết quả từ EX xuống WB.

Giai đoạn MEM bao gồm các khối phần cứng và logic chính sau:

- Data Memory (data_memory) thực hiện đọc/ghi dữ liệu dựa trên tín hiệu MemRead và MemWrite. Khối thực hiện ánh xạ địa chỉ Byte sang chỉ số mảng (dịch phải 1 bit) để truy xuất mảng nhớ, đồng thời kiểm tra biên an toàn trước khi thực hiện lệnh.
- Chuyển tiếp các tín hiệu điều khiển (regWrite, memToReg, w_spec_regs, halt), dữ liệu (alu_result) và địa chỉ thanh ghi đích (write_reg_addr) cho giai đoạn tiếp theo.

4.5.1 Module data_memory

Tín hiệu	Độ rộng	Hướng	Chức năng
clk	1	Input	Xung clock đồng bộ
MemRead	1	Input	Tín hiệu cho phép đọc dữ liệu từ bộ nhớ
MemWrite	1	Input	Tín hiệu cho phép ghi dữ liệu vào bộ nhớ
address	16	Input	Địa chỉ của ô nhớ cần truy cập
write_data	16	Input	Dữ liệu cần ghi vào bộ nhớ
read_data	16	Output	Dữ liệu đọc ra từ bộ nhớ

Bảng 18: *Các tín hiệu của module data_memory*

Hiện thực logic chính:

```
reg [15:0] memory [0:1023];

wire [15:0] word_index;
integer i;

assign word_index = address >> 1;

```

```
    initial begin
        for (i = 0; i < 1024; i = i + 1) begin
            memory[i] = 16'd0;
        end
    end

    always @(*) begin
        if (MemRead) begin
            if (word_index < 1024)
                read_data = memory[word_index];
            else
                read_data = 16'd0;
        end else begin
            read_data = 16'b0;
        end
    end

    always @(posedge clk) begin
        if (MemWrite) begin
            if (word_index < 1024)
                memory[word_index] <= write_data;
        end
    end
```

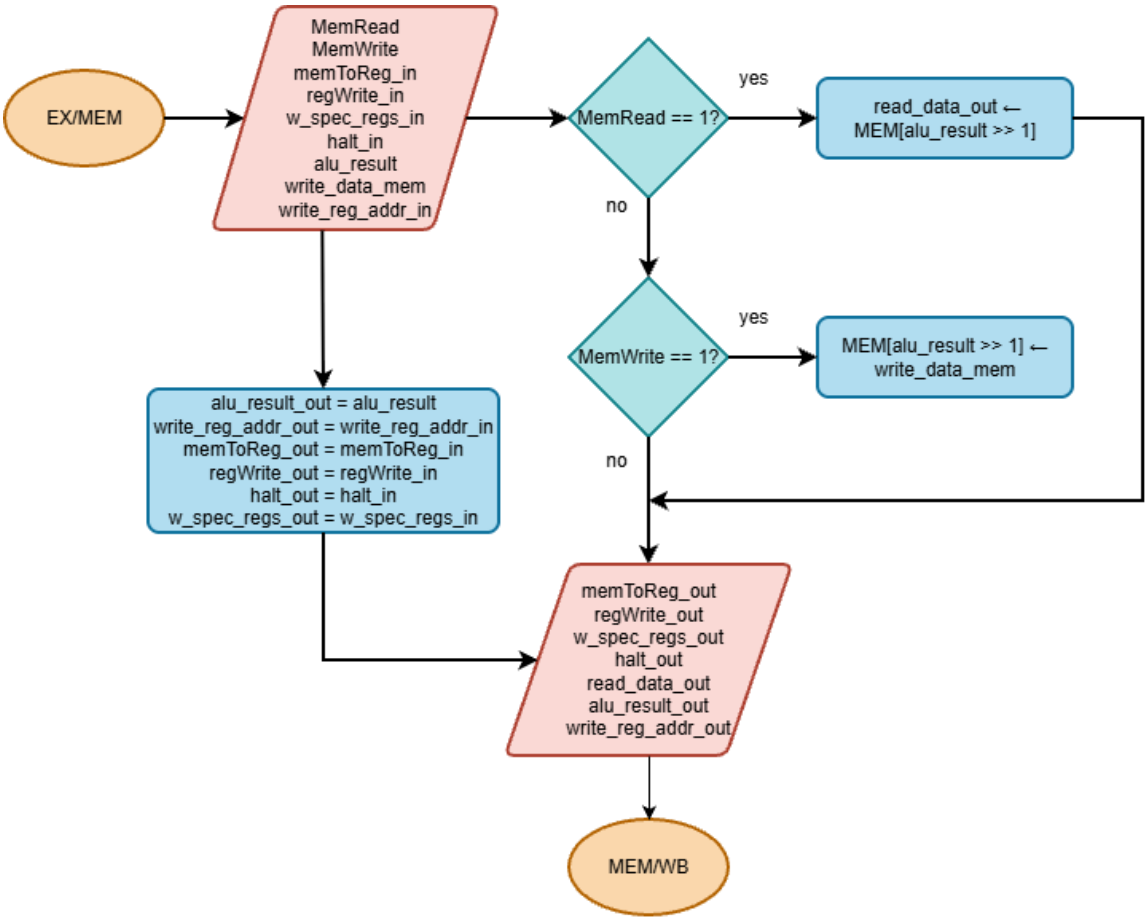
4.5.2 Bảng tín hiệu MEM Stage (Interface)

Tín hiệu	Độ rộng	Hướng	Chức năng
clk	1	Input	Xung clock đồng bộ
MemRead	1	Input	Tín hiệu cho phép đọc dữ liệu từ bộ nhớ
MemWrite	1	Input	Tín hiệu cho phép ghi dữ liệu vào bộ nhớ
memToReg_in	1	Input	Tín hiệu điều khiển nhận từ EX (chuyển tiếp)
regWrite_in	1	Input	Tín hiệu điều khiển nhận từ EX (chuyển tiếp)
w_spec_regs_in	2	Input	Tín hiệu điều khiển nhận từ EX (chuyển tiếp)
halt_in	1	Input	Tín hiệu điều khiển nhận từ EX (chuyển tiếp)
alu_result	16	Input	Kết quả từ ALU (địa chỉ nhớ hoặc dữ liệu tính toán)
write_data_mem	16	Input	Dữ liệu cần ghi vào bộ nhớ
write_reg_addr_in	3	Input	Địa chỉ thanh ghi đích nhận từ EX
read_data_out	16	Output	Dữ liệu đọc ra từ bộ nhớ
alu_result_out	16	Output	Kết quả ALU được chuyển tiếp xuống WB
write_reg_addr_out	3	Output	Địa chỉ thanh ghi đích được chuyển tiếp xuống WB
memToReg_out	1	Output	Tín hiệu được chuyển tiếp sang WB
regWrite_out	1	Output	Tín hiệu được chuyển tiếp sang WB
w_spec_regs_out	2	Output	Tín hiệu được chuyển tiếp sang WB
halt_out	1	Output	Tín hiệu được chuyển tiếp sang WB

Bảng 19: *Các tín hiệu của module MEM_Stage*

4.5.3 Luồng xử lý dữ liệu

Luồng dữ liệu trong giai đoạn MEM bắt đầu từ các tín hiệu nhận được từ tầng EX. Giá trị alu_result được đưa vào hai nhánh: một nhánh đi vào cổng địa chỉ của data_memory để định vị ô nhớ, nhánh còn lại đi ra alu_result_out. Nếu tín hiệu MemWrite tích cực, dữ liệu từ write_data_mem sẽ được ghi vào bộ nhớ tại địa chỉ đã xác định. Ngược lại, nếu MemRead tích cực, dữ liệu từ bộ nhớ được đọc ra read_data_out. Cùng với đó, các tín hiệu điều khiển khác (memToReg, regWrite, w_spec_regs, halt) và địa chỉ thanh ghi đích (write_reg_addr) đi xuyên qua MEM để đến thanh ghi pipeline MEM/WB.



Hình 4.5: *Luồng dữ liệu qua MEM.*

4.6 Thiết kế và hiện thực Write Back

Giai đoạn Write Back (WB) là giai đoạn cuối cùng trong pipeline, có nhiệm vụ cập nhật trạng thái kiến trúc của bộ xử lý dựa trên kết quả thực thi của lệnh. WB stage lựa chọn dữ liệu kết quả cuối cùng và ghi ngược dữ liệu này vào Register File hoặc các thanh ghi đặc biệt tương ứng.

WB stage không tạo ra dữ liệu cho các giai đoạn tiếp theo, mà chỉ hoàn tất vòng đời thực thi của một lệnh trong pipeline. Toàn bộ quá trình ghi dữ liệu trong WB stage được thực hiện đồng bộ tại cạnh lên của xung `clk` nhằm đảm bảo tính nhất quán của pipeline. Về mặt phần cứng, Write Back stage bao gồm các khối chức năng chính sau:

4.6.1 Multiplexer chọn dữ liệu ghi

Module MUX_DataMemory lựa chọn nguồn dữ liệu ghi về thanh ghi, bao gồm kết quả từ ALU hoặc dữ liệu đọc từ Data Memory, dựa trên tín hiệu `memToReg`.

Hướng	Tên tín hiệu	Độ rộng	Mô tả
Input	memtoReg	1 bit	Tín hiệu chọn nguồn dữ liệu ghi.
	read_data	16 bit	Dữ liệu đọc từ Data Memory (đối với các lệnh load).
	alu_result	16 bit	Kết quả tính toán từ ALU.
Output	write_data	16 bit	Dữ liệu cuối cùng được đưa tới Register File để ghi.

Bảng 20: *Giao diện tín hiệu của module Mux_DataMemory*

Nguyên lý hoạt động: MUX_DataMemory hoạt động như một bộ chọn dữ liệu 2–1. Giá trị của tín hiệu `memToReg` quyết định nguồn dữ liệu được truyền ra ngõ ra `write_data`:

- Khi `memToReg = 0`, dữ liệu ghi về thanh ghi được lấy từ kết quả tính toán của ALU (`alu_result`).
- Khi `memToReg = 1`, dữ liệu ghi về thanh ghi được lấy từ bộ nhớ dữ liệu (`read_data`).

Hiện thực logic chính:

```
assign out = (MemtoReg) ? read_data : ALU_result ;
```

4.6.2 Logic điều khiển ghi Register File

Logic điều khiển ghi Register File chịu trách nhiệm quyết định thời điểm và đối tượng được cập nhật dữ liệu trong giai đoạn Write Back. Cơ chế ghi được điều khiển bởi các tín hiệu điều khiển phát sinh từ Control Unit và được truyền qua pipeline đến WB stage.

Tín hiệu `regWrite` đóng vai trò tín hiệu cho phép ghi. Khi `regWrite` được kích hoạt, dữ liệu tại ngõ vào ghi (`write_data`) sẽ được ghi vào thanh ghi đích được chỉ định bởi `write_reg` tại cạnh lên của xung `clk`. Ngược lại, khi `regWrite` không được kích hoạt, Register File giữ nguyên trạng thái hiện tại, không xảy ra cập nhật dữ liệu.

Ghi thanh ghi đặc biệt được xử lý thông qua các tín hiệu điều khiển riêng. Cụ thể, tín hiệu `w_spec_regs` xác định thanh ghi đặc biệt đích trong các lệnh dạng *move to special registers*. Đối với các phép toán số học đặc biệt như nhân và chia, hai thanh ghi `HI` và `LO` được cập nhật độc lập thông qua các tín hiệu `hi_write` và `lo_write`. Các thanh ghi này không thuộc Register File thông thường mà được cài đặt và quản lý riêng trong khối ALU.

- 00: Ghi vào thanh ghi đặc biệt RA
- 01: Ghi vào thanh ghi đặc biệt AT
- 10: Ghi vào thanh ghi đa dụng write_reg

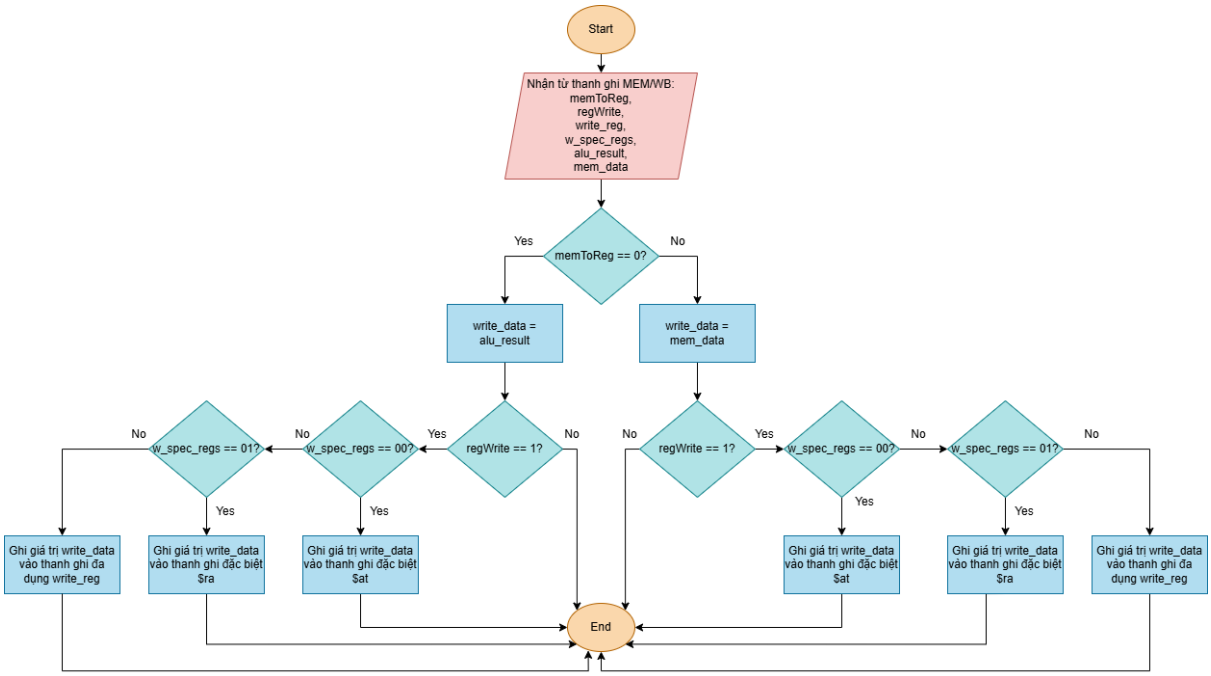
4.6.3 Bảng các tín hiệu WB Stage (Interface)

Hướng	Tên tín hiệu	Độ rộng	Mô tả
Input	alu_result	16 bit	Kết quả tính toán từ ALU.
	mem_data	16 bit	Dữ liệu đọc từ Data Memory.
	write_reg	3 bit	Địa chỉ thanh ghi đích cần ghi dữ liệu.
	memToReg	1 bit	Chọn nguồn dữ liệu ghi (ALU/MEM).
Input	regWrite	1 bit	Cho phép ghi vào Register File.
	w_spec_regs	2 bit	Chọn thanh ghi đặc biệt đích.
	halt	1 bit	Tín hiệu dừng hệ thống.
Output	write_data	16 bit	Dữ liệu cuối cùng được ghi về thanh ghi.
	halt_out	1 bit	Tín hiệu kết thúc chương trình.

Bảng 21: *Giao diện tín hiệu của giai đoạn Write Back*

4.6.4 Luồng xử lý dữ liệu

Kết quả thực thi của lệnh được truyền từ các giai đoạn trước vào WB stage. Multiplexer lựa chọn dữ liệu ghi dựa trên tín hiệu memToReg. Khi tín hiệu cho phép ghi được kích hoạt, dữ liệu sẽ được ghi vào Register File hoặc các thanh ghi đặc biệt tại cạnh lên của xung clk, từ đó hoàn tất quá trình thực thi của lệnh.



Hình 4.6: *Flow chart luồng xử lý của giai đoạn Write Back*

4.7 Thiết kế và hiện thực Pipeline Registers

4.7.1 IF/ID Register

IF/ID Pipeline Register là thanh ghi pipeline đầu tiên trong kiến trúc MIPS 16-bit, đóng vai trò là buffer giữa giai đoạn Instruction Fetch (IF) và Instruction Decode (ID). Module này lưu trữ thông tin của lệnh đã được fetch bao gồm địa chỉ PC, PC+2 và mã lệnh, đảm bảo các giai đoạn pipeline hoạt động độc lập và đồng bộ theo chu kỳ clock.

Tên tín hiệu	Loại	Độ rộng	Mô tả
Control Signals			
clk	Input	1-bit	Tín hiệu clock đồng bộ hệ thống
rst	Input	1-bit	Reset, đặt về trạng thái ban đầu (async)
IF_ID_Write	Input	1-bit	Cho phép ghi: 1=cập nhật, 0=stall
IF_ID_Flush	Input	1-bit	Xóa pipeline, chèn NOP (control hazard)
Inputs từ IF Stage			
IF_pc	Input	16-bit	Địa chỉ PC hiện tại từ IF
IF_pc_plus	Input	16-bit	Địa chỉ PC+2 từ ADD_PC
IF_instruction	Input	16-bit	Lệnh từ Instruction Memory
Outputs sang ID Stage			
ID_pc	Output	16-bit	Địa chỉ PC cho ID stage
ID_pc_plus	Output	16-bit	Địa chỉ PC+2 cho ID stage
ID_instruction	Output	16-bit	Lệnh để giải mã ở ID stage

Bảng 22: *Bảng tín hiệu IF/ID Pipeline Register*

rst	IF_ID_Flush	IF_ID_Write	Hoạt động
1	x	x	Reset: Output ← (0x0000, 0x0000, 0xC000)
0	1	x	Flush: Output ← NOP (chèn bubble)
0	0	1	Write: Output ← Input (normal)
0	0	0	Stall: Output giữ nguyên (data hazard)

Bảng 23: *Bảng trạng thái IF/ID Register*

Nguyên lý hoạt động: IF/ID Register hoạt động đồng bộ với cạnh lên của clock (posedge clk) và được reset bất đồng bộ khi có tín hiệu reset. Logic hoạt động theo thứ tự ưu tiên như sau:

- Reset (ưu tiên cao nhất): Khi rst = 1, tất cả các thanh ghi output được đặt về giá trị khởi tạo. ID_pc và ID_pc_plus được đặt về 0x0000, trong khi ID_instruction được đặt về 0xC000 (mã lệnh NOP). Điều này đảm bảo pipeline bắt đầu từ trạng thái sạch và không có lệnh không hợp lệ được thực thi.
- Flush (ưu tiên thứ hai): Khi IF_ID_Flush = 1, pipeline register bị xóa và chèn bubble (NOP) vào giai đoạn ID. Các giá trị output được đặt giống như trạng thái reset. Cơ chế flush được sử dụng khi có control hazard (branch hoặc jump được thực hiện), cần hủy bỏ các lệnh đã fetch nhưng không nên thực thi.
- Write enable (hoạt động bình thường): Khi IF_ID_Write = 1 và không có reset hay flush, thanh ghi cập nhật dữ liệu mới từ giai đoạn IF. Các giá trị IF_pc, IF_pc_plus và IF_instruction được lưu vào các thanh ghi output tương ứng.
- Stall (giữ nguyên giá trị): Khi IF_ID_Write = 0 (và không có reset hay flush), thanh ghi giữ nguyên giá trị hiện tại. Cơ chế này được sử dụng bởi Hazard Detection Unit để tạm dừng (stall)

pipeline khi phát hiện data hazard, cho phép lệnh phía sau chờ dữ liệu cần thiết.

IF/ID Register đóng vai trò quan trọng trong việc xử lý các loại hazard:

- Control Hazard: Khi có lệnh jump hoặc branch được thực hiện, tín hiệu IF_ID_Flush được kích hoạt để xóa lệnh đã fetch nhầm (wrong path instruction). Lệnh NOP (0xC000) được chèn vào để đảm bảo không có tác động không mong muốn.
- Data Hazard: Khi Hazard Detection Unit phát hiện load-use hazard hoặc các phụ thuộc dữ liệu không thể giải quyết bằng forwarding, IF_ID_Write được đặt về 0. Pipeline register giữ nguyên lệnh hiện tại, tạo ra một chu kỳ stall để chờ dữ liệu sẵn sàng.

IF/ID Register hoạt động đồng bộ hoàn toàn với clock. Tại mỗi cạnh lên của clock, module kiểm tra các điều kiện điều khiển theo thứ tự ưu tiên và cập nhật output tương ứng. Điều này đảm bảo rằng dữ liệu được truyền từ IF sang ID chính xác theo từng chu kỳ clock, duy trì tính đồng bộ của pipeline.

Hiện thực logic chính:

```
always @(posedge clk or posedge rst) begin
    if (rst) begin
        ID_pc <= 16'h0000;
        ID_pc_plus <= 16'h0000;
        ID_instruction <= 16'hc000;
    end
    else if (IF_ID_Flush) begin
        ID_pc <= 16'h0000;
        ID_pc_plus <= 16'h0000;
        ID_instruction <= 16'hc000;
    end
    else if (IF_ID_Write) begin
        ID_pc <= IF_pc;
        ID_pc_plus <= IF_pc_plus;
        ID_instruction <= IF_instruction;
    end
    else begin
        ID_pc <= ID_pc;
        ID_pc_plus <= ID_pc_plus;
        ID_instruction <= ID_instruction;
```

```
        end
    end
```

4.7.2 ID/EX Register

Pipeline register ID/EX là khối thanh ghi trung gian nằm giữa giai đoạn Instruction Decode (ID) và Execute (EX) trong kiến trúc pipeline. Nhiệm vụ của khối này là lưu trữ và đồng bộ toàn bộ tín hiệu điều khiển cũng như dữ liệu cần thiết được tạo ra ở giai đoạn ID để chuyển sang giai đoạn EX ở chu kỳ xung nhịp tiếp theo.

Khối ID/EX register giúp đảm bảo các tín hiệu điều khiển và dữ liệu không bị thay đổi trong suốt quá trình thực thi của lệnh ở giai đoạn EX, đồng thời hỗ trợ cơ chế xử lý hazard thông qua tín hiệu flush khi cần thiết.

Tín hiệu	Độ rộng	Hướng	Chức năng
clk	1	Input	Xung nhịp hệ thống
rst	1	Input	Reset toàn bộ pipeline register
ID_EX_Flush	1	Input	Xóa dữ liệu ID/EX khi xảy ra hazard hoặc branch
ID_ALU_cnt	6	Input	Tín hiệu điều khiển ALU từ giai đoạn ID
ID_PCSrc	2	Input	Điều khiển nguồn cập nhật PC
ID_regDst	1	Input	Chọn thanh ghi đích (Rt hoặc Rd)
ID_ALUSrc	1	Input	Chọn toán hạng thứ hai cho ALU
ID_memWrite	1	Input	Cho phép ghi bộ nhớ
ID_memRead	1	Input	Cho phép đọc bộ nhớ
ID_regWrite	1	Input	Cho phép ghi thanh ghi
ID_memToReg	1	Input	Chọn dữ liệu ghi về thanh ghi
ID_r_spec_regs	2	Input	Xác định thanh ghi đặc biệt được đọc
ID_w_spec_regs	2	Input	Xác định thanh ghi đặc biệt được ghi
ID_halt	1	Input	Dừng hoạt động pipeline
ID_pc	16	Input	Giá trị PC hiện tại
ID_pc_plus	16	Input	Giá trị PC + 1
ID_rd1	16	Input	Dữ liệu đọc từ thanh ghi nguồn Rs
ID_rd2	16	Input	Dữ liệu đọc từ thanh ghi nguồn Rt
ID_sign_ext	16	Input	Giá trị immediate sau sign extension
ID_br_target	16	Input	Địa chỉ nhánh (branch target)
ID_rs, ID_rt, ID_rd	3	Input	Các trường thanh ghi của lệnh
EX_*	-	Output	Các tín hiệu được truyền sang giai đoạn EX

Bảng 24: *Bảng mô tả tín hiệu Pipeline Register ID/EX*

Nguyên lý hoạt động: Pipeline register ID/EX hoạt động đồng bộ theo cạnh lên của xung nhịp `clk`. Khi tín hiệu `rst` hoặc `ID_EX_Flush` được kích hoạt, toàn bộ các tín hiệu điều khiển và dữ liệu bên trong pipeline register sẽ được đưa về trạng thái mặc định (NOP), nhằm ngăn chặn việc thực thi các lệnh không hợp lệ.

Trong điều kiện hoạt động bình thường, tại mỗi cạnh lên của xung nhịp, ID/EX register sẽ chốt toàn bộ tín hiệu điều khiển và dữ liệu từ giai đoạn ID và truyền sang giai đoạn EX.

Thanh ghi đích được xác định dựa trên tín hiệu `ID_regDst`. Nếu `ID_regDst = 1`, thanh ghi đích là `Rd`; ngược lại, thanh ghi đích là `Rt`. Giá trị này được lưu vào `EX_rd` để sử dụng trong các giai đoạn tiếp theo của pipeline.

Việc sử dụng cơ chế flush giúp pipeline xử lý hiệu quả các tình huống hazard và branch, đảm bảo tính đúng đắn của quá trình thực thi lệnh.

Hiện thực logic chính:

```
wire [2:0] destination_reg = ID_regDst ? ID_rd : ID_rt;
always @(posedge clk or posedge rst) begin
    if (rst || ID_EX_Flush) begin
        EX_ALU_cnt <= 6'b111111;
        EX_PCSrc <= 2'b00;
        EX_r_spec_regs <= 2'b11;
        EX_w_spec_regs <= 2'b10;
        EX_regDst <= 1'b0;
        EX_ALUSrc <= 1'b0;
        EX_memWrite <= 1'b0;
        EX_memRead <= 1'b0;
        EX_regWrite <= 1'b0;
        EX_memToReg <= 1'b0;
        EX_halt <= 1'b0;

        EX_pc <= 16'h0000;
        EX_pc_plus <= 16'h0000;
        EX_rd1 <= 16'h0000;
        EX_rd2 <= 16'h0000;
        EX_sign_ext <= 16'h0000;
```

```
            EX_br_target <= 16'h0000;
            EX_rs <= 3'b000;
            EX_rt <= 3'b000;
            EX_rd <= 3'b000;
        end

        else begin
            EX_ALU_cnt <= ID_ALU_cnt;
            EX_PCSrc <= ID_PCSrc;
            EX_r_spec_regs <= ID_r_spec_regs;
            EX_w_spec_regs <= ID_w_spec_regs;
            EX_regDst <= ID_regDst;
            EX_ALUSrc <= ID_ALUSrc;
            EX_memWrite <= ID_memWrite;
            EX_memRead <= ID_memRead;
            EX_regWrite <= ID_regWrite;
            EX_memToReg <= ID_memToReg;
            EX_halt <= ID_halt;

            EX_pc <= ID_pc;
            EX_pc_plus <= ID_pc_plus;
            EX_rd1 <= ID_rd1;
            EX_rd2 <= ID_rd2;
            EX_sign_ext <= ID_sign_ext;
            EX_br_target <= ID_br_target;
            EX_rs <= ID_rs;
            EX_rt <= ID_rt;
            EX_rd <= destination_reg;
      end
    end
```

4.7.3 EX/MEM Register

Pipeline register EX/MEM là khối thanh ghi trung gian nằm giữa giai đoạn Execute (EX) và Memory Access (MEM) trong kiến trúc pipeline của CPU RISC 16-bit dựa trên MIPS.

Nhiệm vụ chính của EX/MEM register là lưu trữ kết quả xử lý từ ALU, các tín hiệu điều khiển liên quan đến truy cập bộ nhớ và ghi thanh ghi, đồng thời chuyển các giá trị này sang giai đoạn MEM ở chu kỳ xung nhịp kế tiếp.

Khối EX/MEM đảm bảo sự đồng bộ giữa các giai đoạn pipeline và giúp quá trình truy cập bộ nhớ dữ liệu cũng như xử lý lệnh nhánh được thực hiện chính xác.

Tín hiệu	Độ rộng	Hướng	Chức năng
clk	1	Input	Xung nhịp hệ thống
rst	1	Input	Reset pipeline register EX/MEM
EX_memWrite	1	Input	Cho phép ghi bộ nhớ dữ liệu
EX_memRead	1	Input	Cho phép đọc bộ nhớ dữ liệu
EX_regWrite	1	Input	Cho phép ghi về thanh ghi
EX_memtoReg	1	Input	Chọn dữ liệu ghi về thanh ghi
EX_w_spec_regs	2	Input	Xác định thanh ghi đặc biệt được ghi
EX_halt	1	Input	Dừng hoạt động pipeline
EX_pc	16	Input	Giá trị PC của lệnh đang thực thi
EX_pc_plus	16	Input	Giá trị PC + 1
EX_alu_result	16	Input	Kết quả tính toán từ ALU
EX_rd2	16	Input	Dữ liệu từ thanh ghi Rt (phục vụ lệnh store)
EX_write_reg	3	Input	Thanh ghi đích của lệnh
MEM_*	-	Output	Các tín hiệu được truyền sang giai đoạn MEM

Bảng 25: *Bảng mô tả tín hiệu Pipeline Register EX/MEM*

Nguyên lý hoạt động: Pipeline register EX/MEM hoạt động đồng bộ theo cạnh lên của xung nhịp `clk`. Khi tín hiệu `rst` được kích hoạt, toàn bộ các tín hiệu điều khiển và dữ liệu bên trong EX/MEM register sẽ được đưa về giá trị mặc định nhằm đảm bảo pipeline không thực thi các lệnh không hợp lệ.

Trong chế độ hoạt động bình thường, tại mỗi cạnh lên của xung nhịp, EX/MEM register sẽ chốt các tín hiệu điều khiển và dữ liệu đầu ra từ giai đoạn EX, bao gồm kết quả ALU, dữ liệu phục vụ truy cập bộ nhớ và thông tin thanh ghi đích.

Các tín hiệu `MEM_memRead` và `MEM_memWrite` được sử dụng trực tiếp trong giai đoạn MEM để điều khiển bộ nhớ dữ liệu, trong khi `MEM_regWrite` và `MEM_memtoReg` được chuyển tiếp sang giai đoạn WB để ghi

kết quả về thanh ghi.

Hiện thực logic chính:

```
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            MEM_memWrite <= 1'b0;
            MEM_memRead <= 1'b0;
            MEM_regWrite <= 1'b0;
            MEM_memtoReg <= 1'b0;
            MEM_w_spec_regs <= 2'b10;
            MEM_halt <= 1'b0;

            MEM_pc <= 16'h0000;
            MEM_pc_plus <= 16'h0000;
            MEM_alu_result <= 16'h0000;
            MEM_rd2 <= 16'h0000;
            MEM_write_reg <= 3'b000;
        end

        else begin
            MEM_memWrite <= EX_memWrite;
            MEM_memRead <= EX_memRead;
            MEM_regWrite <= EX_regWrite;
            MEM_memtoReg <= EX_memtoReg;
            MEM_w_spec_regs <= EX_w_spec_regs;
            MEM_halt <= EX_halt;

            MEM_pc <= EX_pc;
            MEM_pc_plus <= EX_pc_plus;
            MEM_alu_result <= EX_alu_result;
            MEM_rd2 <= EX_rd2;
            MEM_write_reg <= EX_write_reg;
        end
    end
```

4.7.4 MEM/WB Register

Pipeline register MEM/WB là khối thanh ghi trung gian nằm giữa giai đoạn Memory Access (MEM) và Write Back (WB) trong kiến trúc pipeline của CPU RISC 16-bit dựa trên MIPS.

Chức năng chính của MEM/WB register là lưu trữ dữ liệu đọc từ bộ nhớ, kết quả tính toán từ ALU và các tín hiệu điều khiển cần thiết cho giai đoạn WB. Các giá trị này được chốt lại theo từng chu kỳ xung nhịp và chuyển sang giai đoạn ghi kết quả về thanh ghi.

Khối MEM/WB đóng vai trò quan trọng trong việc đảm bảo dữ liệu và tín hiệu điều khiển được đồng bộ, giúp quá trình ghi kết quả về thanh ghi diễn ra chính xác và ổn định.

Tín hiệu	Độ rộng	Hướng	Chức năng
clk	1	Input	Xung nhịp hệ thống
rst	1	Input	Reset pipeline register MEM/WB
regWrite_in	1	Input	Cho phép ghi dữ liệu về thanh ghi
memToReg_in	1	Input	Chọn nguồn dữ liệu ghi về thanh ghi
w_spec_regs_in	2	Input	Điều khiển ghi thanh ghi đặc biệt (HI/LO)
halt_in	1	Input	Tín hiệu dừng CPU
read_data_in	16	Input	Dữ liệu đọc từ bộ nhớ dữ liệu
alu_result_in	16	Input	Kết quả tính toán từ ALU
write_reg_addr_in	3	Input	Địa chỉ thanh ghi đích
regWrite_out	1	Output	Điều khiển ghi thanh ghi tại WB
memToReg_out	1	Output	Chọn dữ liệu ghi về thanh ghi
w_spec_regs_out	2	Output	Điều khiển ghi thanh ghi đặc biệt
halt_out	1	Output	Dừng pipeline
read_data_out	16	Output	Dữ liệu đọc từ bộ nhớ sang WB
alu_result_out	16	Output	Kết quả ALU sang WB
write_reg_addr_out	3	Output	Địa chỉ thanh ghi đích

Bảng 26: *Bảng mô tả tín hiệu Pipeline Register MEM/WB*

Nguyên lý hoạt động: Pipeline register MEM/WB hoạt động đồng bộ theo cạnh lên của xung nhịp `clk`. Khi tín hiệu `rst` được kích hoạt, toàn bộ các tín hiệu điều khiển và dữ liệu trong khối MEM/WB sẽ được đặt về giá trị mặc định nhằm tránh việc ghi sai dữ liệu về thanh ghi.

Trong quá trình hoạt động bình thường, tại mỗi cạnh lên của xung nhịp, MEM/WB register chốt các tín hiệu điều khiển từ giai đoạn MEM, bao gồm `regWrite`, `memToReg` và thông tin ghi thanh ghi đặc biệt.

Dữ liệu đọc từ bộ nhớ (`read_data_in`) và kết quả từ ALU (`alu_result_in`) được lưu trữ để cung cấp

cho giai đoạn WB. Dựa trên tín hiệu `memToReg`, khối Write Back sẽ lựa chọn nguồn dữ liệu phù hợp để ghi về thanh ghi đích được xác định bởi `write_reg_addr_out`.

Nhờ pipeline register MEM/WB, quá trình ghi kết quả được tách biệt hoàn toàn với giai đoạn truy cập bộ nhớ, giúp tăng hiệu suất và đảm bảo tính ổn định cho hệ thống.

Hiện thực logic chính:

```
    always @(posedge clk or posedge rst) begin
        if (rst) begin
            regWrite_out <= 0;
            memToReg_out <= 0;
            halt_out <= 0;
            read_data_out <= 0;
            alu_result_out <= 0;
            write_reg_addr_out <= 0;
            w_spec_regs_out <= 2'b10;
        end else begin
            regWrite_out <= regWrite_in;
            memToReg_out <= memToReg_in;
            halt_out <= halt_in;

            read_data_out <= read_data_in;
            alu_result_out <= alu_result_in;
            write_reg_addr_out <= write_reg_addr_in;
            w_spec_regs_out <= w_spec_regs_in;
        end
    end
```

5 Kiểm thử và đánh giá

5.1 Kiểm thử

Để xác minh tính đúng đắn và độ tin cậy của thiết kế RISC CPU 16-bit, nhóm đã áp dụng quy trình kiểm thử hai giai đoạn: kiểm tra từng module riêng lẻ (nhóm đã thực hiện trong quá trình hiện thực từng module) và kiểm tra tích hợp trên toàn bộ hệ thống hoàn chỉnh. Bộ kịch bản kiểm thử được xây dựng

để kiểm tra toàn bộ các nhóm chức năng, bao gồm: các phép toán số học/logic (ALU0, ALU1, ALU2, Immediate), thao tác thanh ghi đặc biệt (MTSR, MFSR), thao tác truy xuất bộ nhớ (Load/Store), điều khiển luồng (Branch/Jump) và đặc biệt là khả năng xử lý các xung đột đường ống (Pipeline Hazards).

Để thực hiện kiểm thử, đưa các tín hiệu sau ra output của module TOP:

Tín hiệu	**Độ rộng**	**Chức năng trong Mô phỏng**
pc_current	16	Giá trị PC tại IF. Dùng để theo dõi luồng thực thi lệnh và kiểm tra tính đúng đắn của các lệnh nhảy/rẽ nhánh trên waveform
result_final	16	Kết quả tính toán tại tầng WB. Dùng để kiểm tra tính đúng đắn của lệnh
write_reg	4	Địa chỉ thanh ghi đích. Dùng để xác thực lệnh có ghi đúng thanh ghi đích không
spec_reg_id	2	Trạng thái ghi các thanh ghi đặc biệt. Kiểm tra có đang ghi vào thanh ghi đặc biệt không

Bảng 27: *Các output được in ra để kiểm thử*

Tại thời điểm khởi chạy mô phỏng ($t = 0$), tín hiệu `rst` được kích hoạt mức cao (Logic 1) để khởi tạo trạng thái ban đầu cho tất cả các thanh ghi và bộ đếm chương trình (PC). Trạng thái này được duy trì trong 20ns để đảm bảo hệ thống ổn định, sau đó tín hiệu `rst` được hạ xuống mức thấp (Logic 0) để CPU bắt đầu chu trình thực thi lệnh.

5.1.1 Nhóm lệnh Immediate

Mục tiêu: Kiểm tra khả năng xử lý các lệnh Immediate.

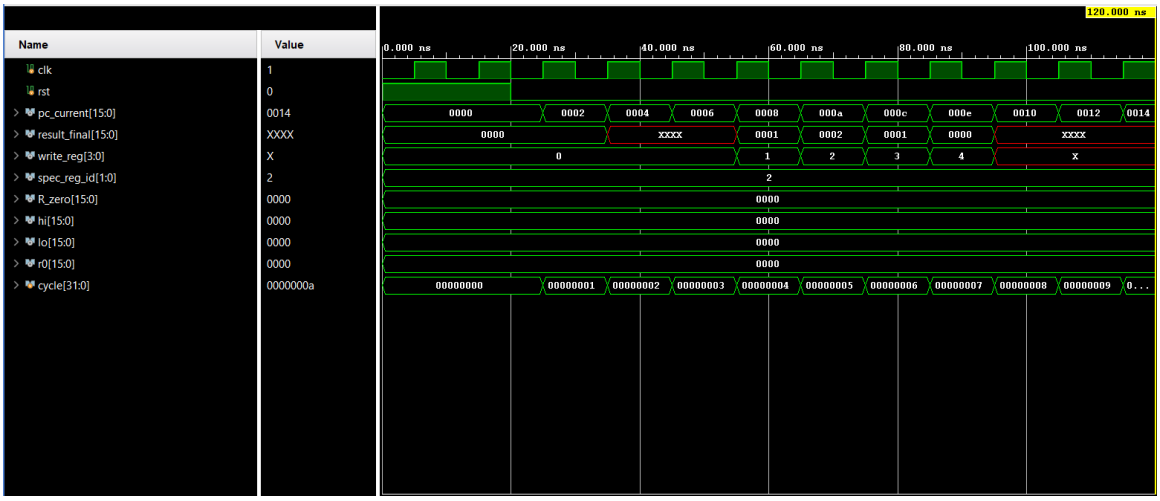
Program 1: Test Case Immediate

```
addi $1, $7, 1    // $1 = 0 + 1 = 1
addi $2, $7, 2    // $2 = 0 + 2 = 2
slti $3, $2, 3    // $3 = (2 < 3) ? 1 : 0
slti $4, $2, 1    // $3 = (2 < 1) ? 1 : 0
```

Kết quả mong đợi:

- Thanh ghi $1 = 1, $2 = 2.
- Thanh ghi $3 = 1 (True).
- Thanh ghi $4 = 0 (False).

Kết quả mô phỏng trên Vivado:



Hình 5.1: *Kết quả mô phỏng cho các lệnh Immediate.*

5.1.2 Nhóm lệnh ALU0

Mục tiêu: Kiểm tra các phép toán không dấu và logic.

Program 2: Test Case ALU0

```
// Add/Sub
addi $1, $7, 1        // $1 = 1
addi $2, $7, 2        // $2 = 2
addu $3, $1, $2       // $3 = $1 + $2 = 1 + 2 = 3
subu $3, $2, $1       // $3 = $2 - $1 = 2 - 1 = 1
subu $3, $1, $2       // $3 = $1 - $2 = 1 - 2 = ffff (unsigned)

// Mult/Div
addi $1, $7, 31       // $1 = 31
addi $2, $7, 29       // $2 = 29
multu $1, $2          // $1 * $2 = 31 * 29 = 899 (0x0383)
mfhi $4               // $4 = [$HI]
mflo $4               // $4 = [$LO]
divu $1, $2           // $1 / $2 = 31 / 29 = 1 (Remainder 2)
mfhi $4               // $4 = [$HI]
mflo $4               // $4 = [$LO]
```

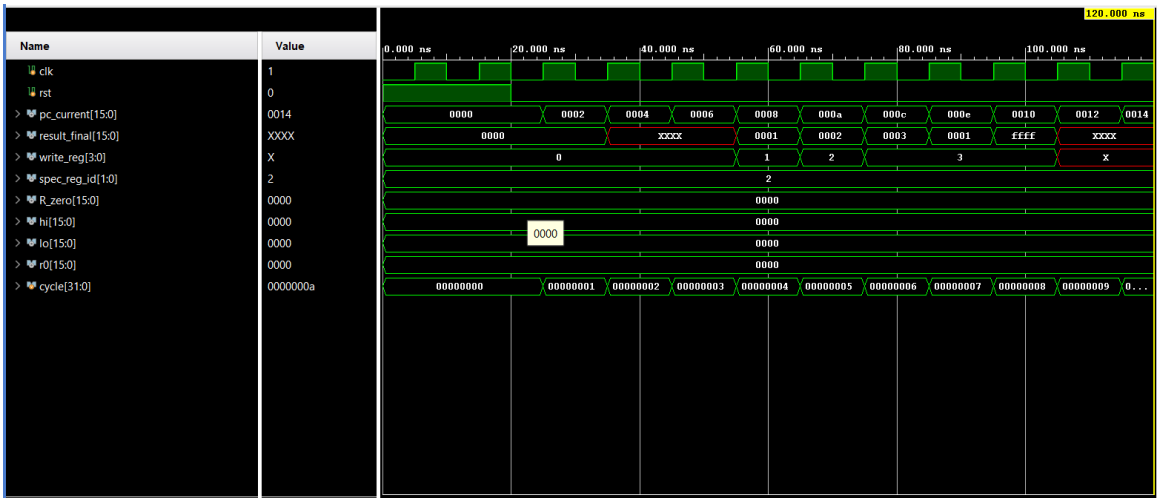
```

// Logical
addi $1, $7, 14      // $1 = 14
addi $2, $7, 5       // $2 = 5
and $3, $1, $2       // $3 = $1 & $2
or $3, $1, $2        // $3 = $1 | $2
nor $3, $1, $2       // $3 = ~ ($1 | $2)
xor $3, $1, $2       // $3 = $1 xor $2
```

Kết quả mong đợi:

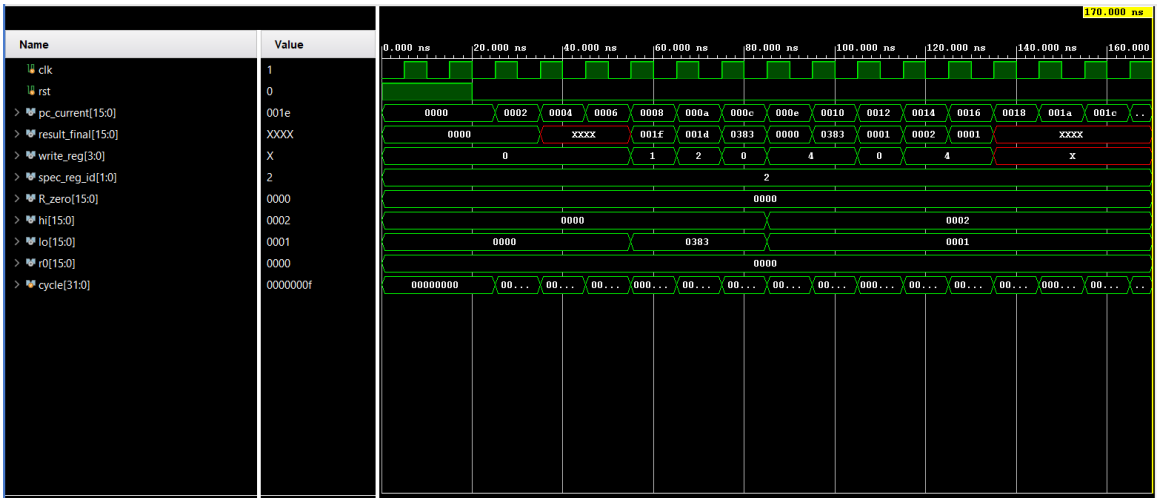
- Phép cộng/trừ: đúng kết quả tính toán thực tế.
- Phép nhân: Thanh ghi HI = 0, LO = 899 (0x0383).
- Phép chia: Thanh ghi HI = 2, LO = 1.
- Các phép Logic:

Lệnh	Phép toán	Tính toán (Nhị phân 4-bit)	Kết quả (Hex)
and	$14 \,\&\, 5$	$1110 \cdot 0101 = 0100$	0x0004
or	$14 \mid 5$	$1110 + 0101 = 1111$	0x000F
nor	$\sim (14 \mid 5)$	$\sim (\ldots 001111) = \ldots 110000$	0xFFF0
xor	$14 \oplus 5$	$1110 \oplus 0101 = 1011$	0x000B

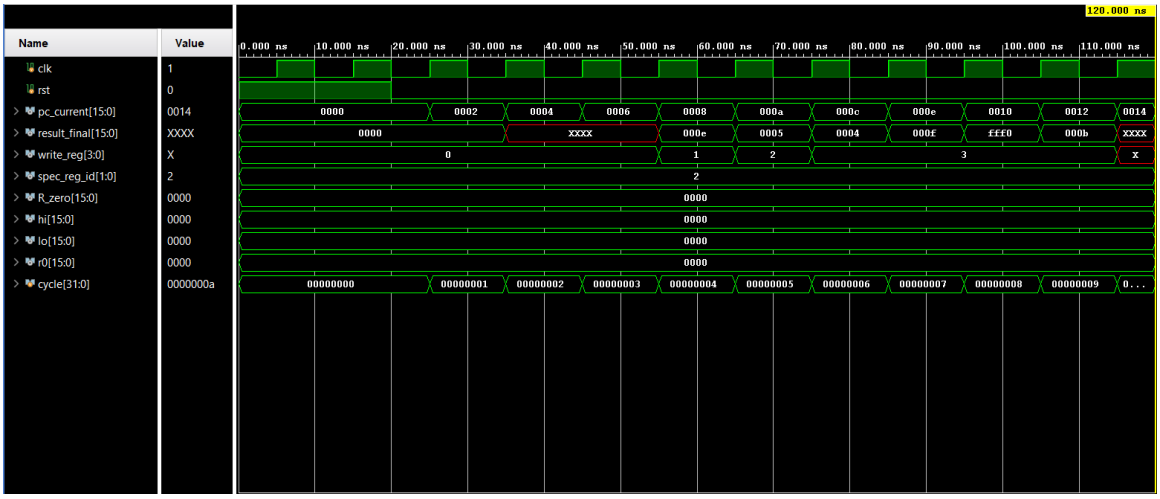
Kết quả mô phỏng trên Vivado:



Hình 5.2: *Kết quả mô phỏng cho các lệnh addu, subu.*



Hình 5.3: *Kết quả mô phỏng cho các lệnh multu, divu.*



Hình 5.4: *Kết quả mô phỏng cho các lệnh Logical.*

5.1.3 Nhóm lệnh ALU1

Mục tiêu: Kiểm tra tính toán số học có dấu và các cờ so sánh.

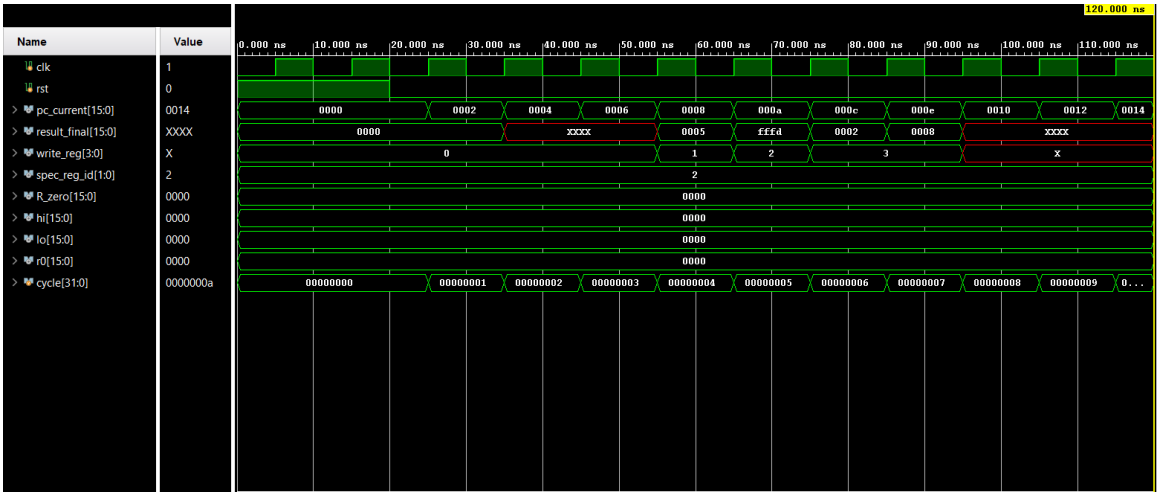
Program 3: Test Case ALU1

```
// Add/Sub
addi $1, $7, 5       // $1 = 5
addi $2, $7, -3      // $2 = -3
add $3, $1, $2       // $3 = $1 + $2 = 5 + (-3) = 2
sub $3, $1, $2       // $3 = $1 + $2 = 5 - (-3) = 8

// Mult/Div
addi $1, $7, 5       // $1 = 5
addi $2, $7, -3      // $2 = -3 (0xfffd)
mult $1, $2         // $1 * $2 = 5 * (-3) = -15
mfhi $4              // $4 = [$HI]
mflo $4              // $4 = [$LO]
div $1, $2          // $1 / $2 = 5 / (-3) = -1 (Remainder 2)
mfhi $4              // $4 = [$HI]
mflo $4              // $4 = [$LO]

// Compare
addi $1, $7, 5       // $1 = 5
addi $2, $7, -10     // $2 = -10
addi $5, $7, 5       // $5 = 5
slt $3, $1, $2       // $3 = (5 < (-10)) ? 1 : 0 = 0
slt $3, $2, $1       // $3 = ((-10) < 5) ? 1 : 0 = 1
seq $3, $1, $2       // $3 = (5 == (-10)) ? 1 : 0 = 0
seq $3, $1, $5       // $3 = (5 == 5 ? 1 : 0 = 1
sltu $3, $1, $2      // $3 = (5 < (-10)) ? 1 : 0 = 1 (unsigned)
sltu $3, $2, $1      // $3 = ((-10) < 5) ? 1 : 0 = 0 (unsigned)

// Jump Register
addi $1, $7, 2       // $1 = 2
jr $1                // Jump to the address in $1
```

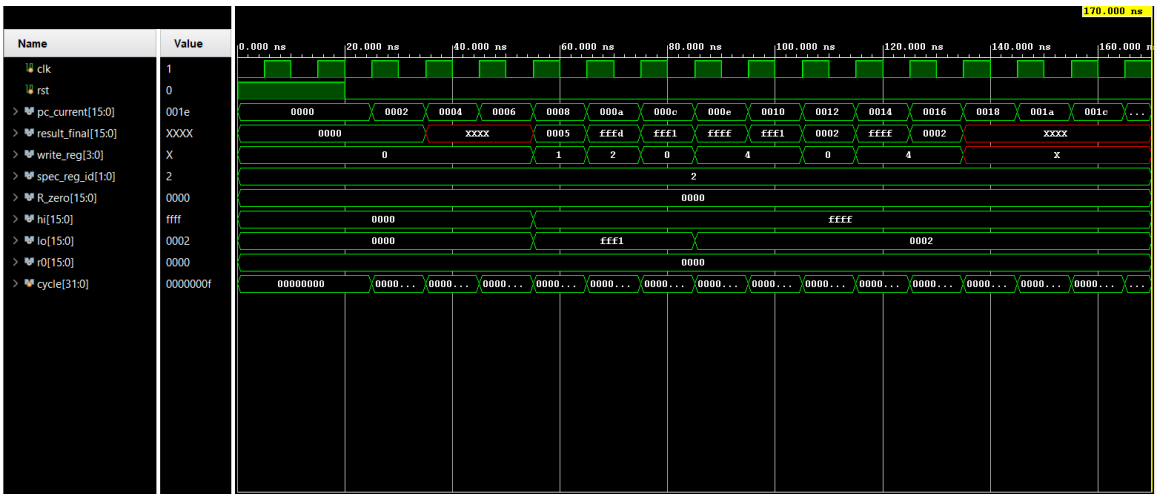
Kết quả mong đợi:

- Phép tính có dấu xử lý đúng bù 2.
- Phép nhân: Thanh ghi HI = 0xffff, LO = 0xfff1.
- Phép chia: Thanh ghi HI = 0xffff, LO = 2.
- Phép so sánh: so sánh đúng các số có dấu.
- Lệnh `jr` cập nhật PC thành 2.

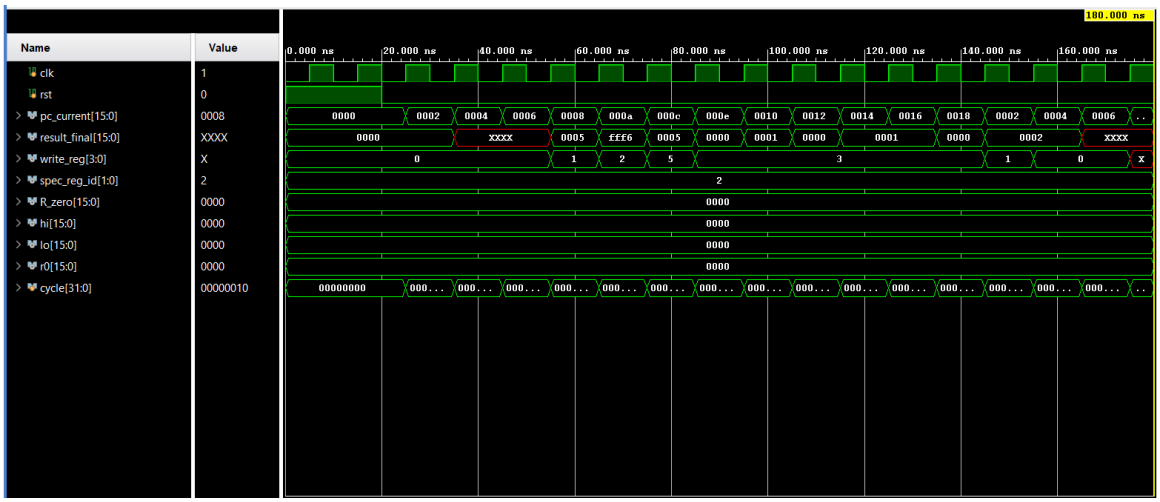
Kết quả mô phỏng trên Vivado:



Hình 5.5: *Kết quả mô phỏng cho các lệnh add, sub.*



Hình 5.6: *Kết quả mô phỏng cho các lệnh mult, div.*



Hình 5.7: *Kết quả mô phỏng cho các lệnh Compare và jr.*

5.1.4 Nhóm lệnh ALU2

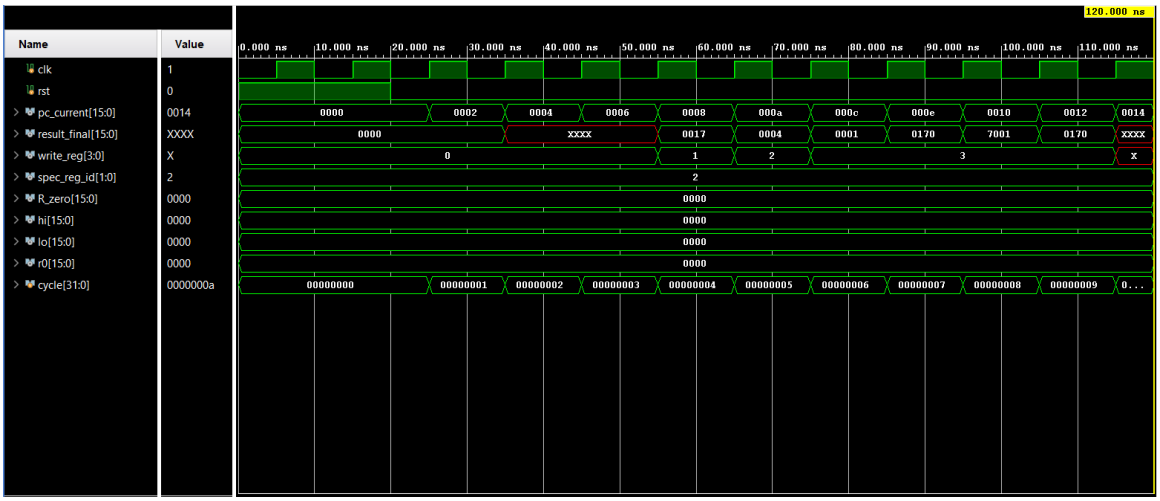
Mục tiêu: Kiểm tra bộ dịch bit.

Program 4: Test Case Shifter

```
addi $1, $7, 23      // $1 = 23 (0x0017)
addi $2, $0, 4       // $2 = 4
shr $3, $2, $1       // Shift Right: $3 = 0x0001
shl $3, $2, $1       // Shift Left: $3 = 0x0170
ror $3, $2, $1       // Rotate Right: $3 = 0x7001
rol $3, $2, $1       // Rotate Left: $3 = 0x0170
```

Kết quả mong đợi: Dữ liệu được dịch đúng số bit quy định.

Kết quả mô phỏng trên Vivado:



Hình 5.8: *Kết quả mô phỏng cho các lệnh ALU2.*

5.1.5 Các lệnh MTSR/MFSR

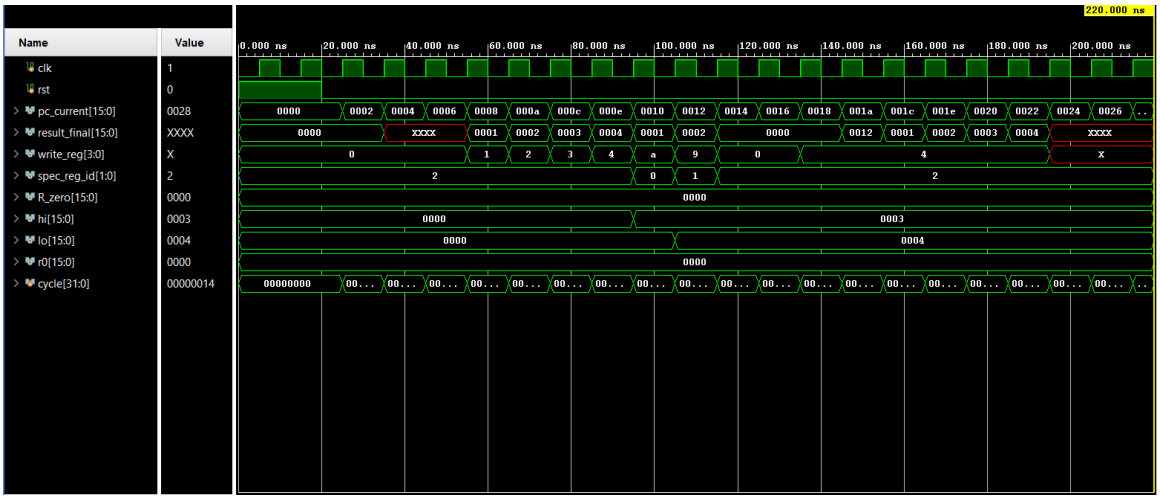
Mục tiêu: Kiểm tra các lệnh liên quan đến thanh ghi đặc biệt

Program 5: Test Case MTSR/MFSR

```
addi $1, $7, 1        // $1 = 1
addi $2, $7, 2        // $1 = 1
addi $3, $7, 3        // $1 = 1
addi $4, $7, 4        // $1 = 1

mtra $1               // Move to RA
mtat $2               // Move to AR
mthi $3               // Move to HI
mtlo $4               // Move to LO

mfz $4                // Move from ZERO
mfpc $4               // Move from PC
mfra $4               // Move from RA
mtat $2               // Move from AR
mthi $3               // Move from HI
mtlo $4               // Move from LO
```

Kết quả mong đợi: Dữ liệu ghi vào và đọc ra khớp nhau.

Kết quả mô phỏng trên Vivado:



Hình 5.9: *Kết quả mô phỏng cho các lệnh MTSR, MFSR.*

5.1.6 Các lệnh Memory & Load-Use Hazard

Mục tiêu: Kiểm tra lệnh Load/Store và khả năng Stall khi có xung đột.

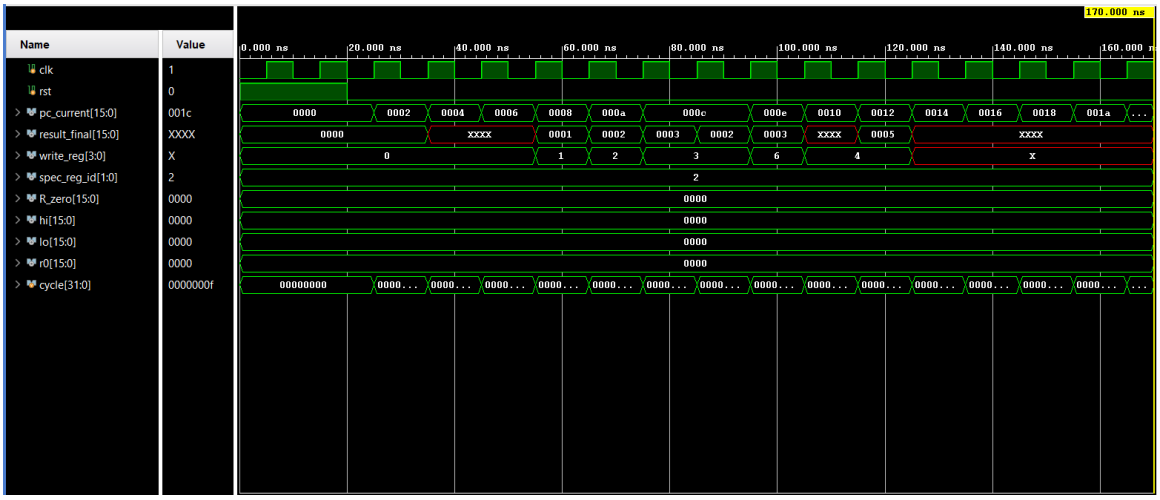
Program 6: Test Case Memory and Load-Use Hazard

```
addi $1, $7, 1        // $1 = 1
addi $2, $7, 2        // $2 = 2
add $3, $2, $1        // $3 = 3
sh $3, 0($2)          // Store $3 into Mem
lh $6, 0($2)          // Load Mem into $6
addi $4, $6, 2        // Use $6 right after loading (Hazard)
```

Kết quả mong đợi:

- CPU tự động chèn 1 chu kỳ Stall (bong bóng) giữa lệnh `lh` và `addi`.
- Kết quả cuối cùng: $\$6 = 3$ và $\$4 = 5$.

Kết quả mô phỏng trên Vivado:



Hình 5.10: *Kết quả mô phỏng cho các lệnh Memory và Load-Use Hazard.*

5.1.7 Các lệnh Branch & Jump

Mục tiêu: Kiểm tra rẽ nhánh và nhảy.

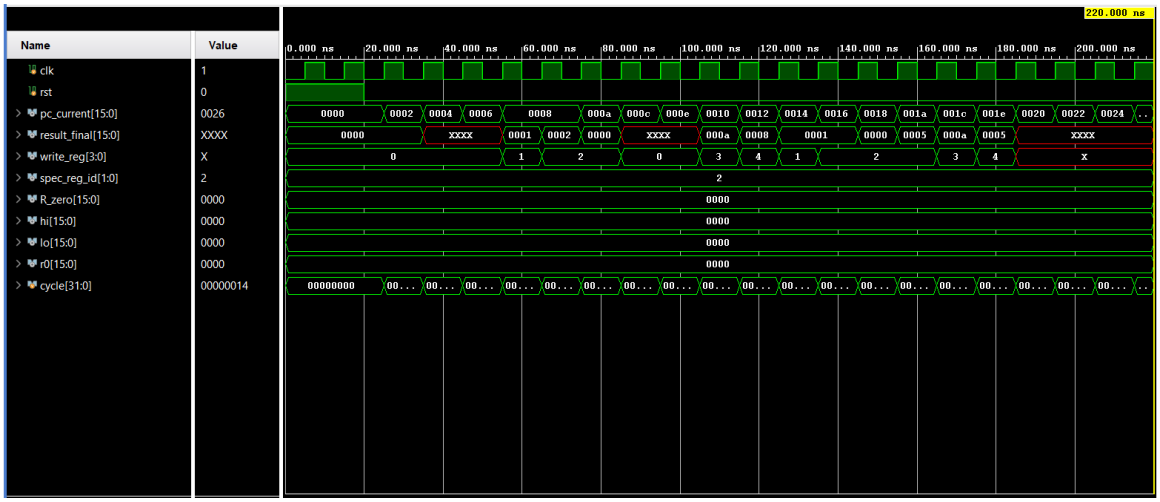
Program 7: Test Case Branch and Jump

```
// bneq
addi $1, $7, 1        // $1 = 1
addi $2, $7, 2        // $2 = 2
bneq $1, $2, 2        // If $1 != $2, skip 2 commands (true)
addi $2, $7, 5        // flush
addi $3, $7, 10       // $3 = 10
sub $4, $3, $2        // $4 = 10 - 2 = 8
addi $1, $7, 1        // $1 = 1
addi $2, $7, 1        // $2 = 1
bneq $1, $2, 2        // If $1 != $2, skip 2 commands (false)
addi $2, $7, 5        // $2 = 5
addi $3, $7, 10       // $3 = 10
sub $4, $3, $2        // $4 = 10 - 5 = 5

// bgtz
addi $1, $7, 1        // $1 = 1
bgtz $1, 2            // If $1 > 0, skip 2 commands (true)
addi $2, $7, 5        // flush
```

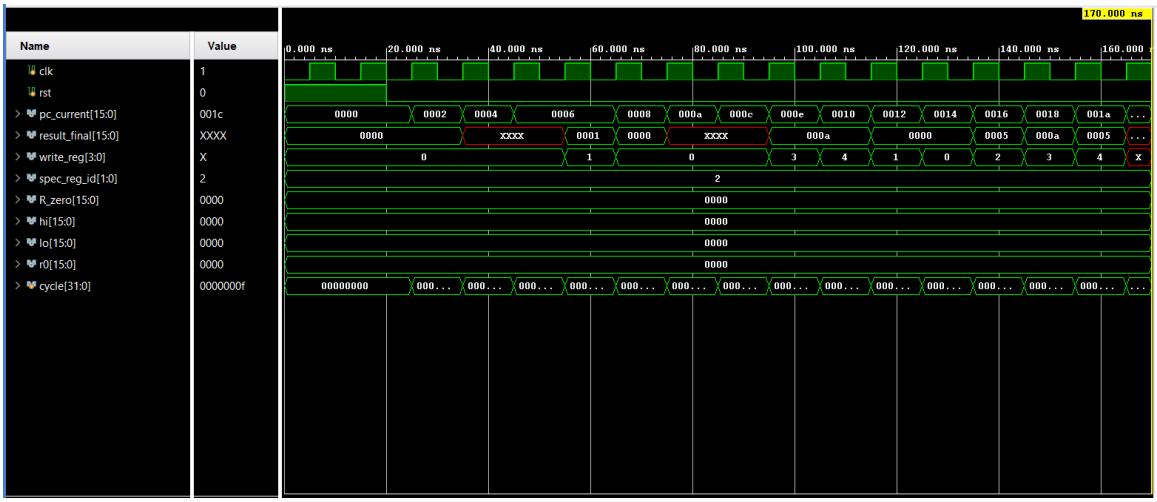
```
addi $3, $7, 10      // $3 = 10
sub $4, $3, $2       // $4 = 10 - 0 = 10
addi $1, $7, 0       // $1 = 0
bgtz $1, 2           // If $1 > 0, skip 2 commands (false)
addi $2, $7, 5       // $2 = 5
addi $3, $7, 10      // $3 = 10
sub $4, $3, $2       // $4 = 10 - 5 = 5

// Jump
addi $1, $7, 1       // $1 = 1
addi $2, $7, 2       // $2 = 2
j 4                  // Jump unconditionally to command 4
add $2, $1, $2
add $3, $2, $1       // Jump here: $3 = 2 + 1 = 3
```

Kết quả mong đợi: PC cập nhật đúng địa chỉ đích. Các lệnh bị nhảy qua không được thực thi (hoặc bị flush).

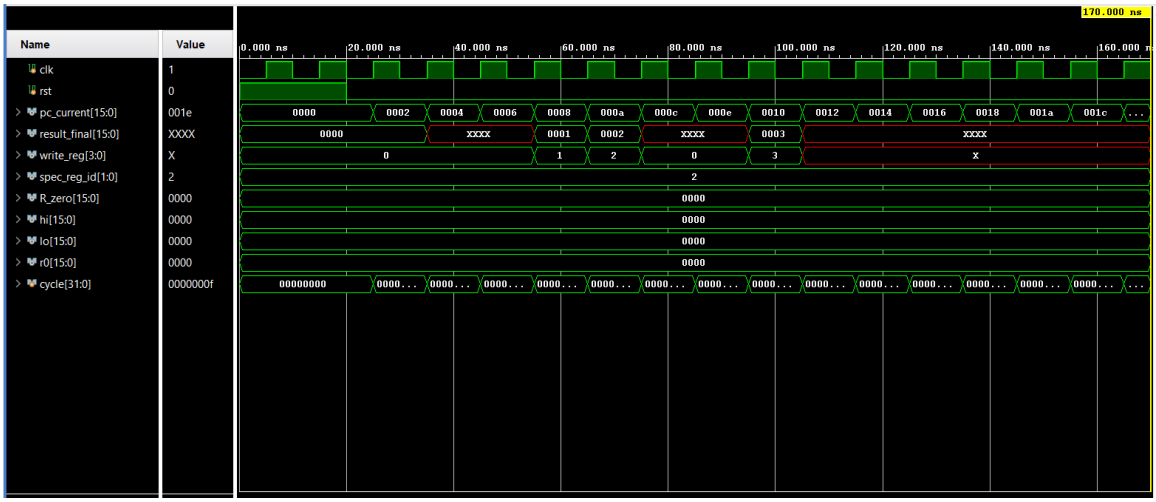
Kết quả mô phỏng trên Vivado:



Hình 5.11: *Kết quả mô phỏng cho lệnh bneq.*



Hình 5.12: *Kết quả mô phỏng cho lệnh bgtz.*



Hình 5.13: *Kết quả mô phỏng cho lệnh Jump.*

5.1.8 Các lệnh Data Hazard (RAW - Read After Write)

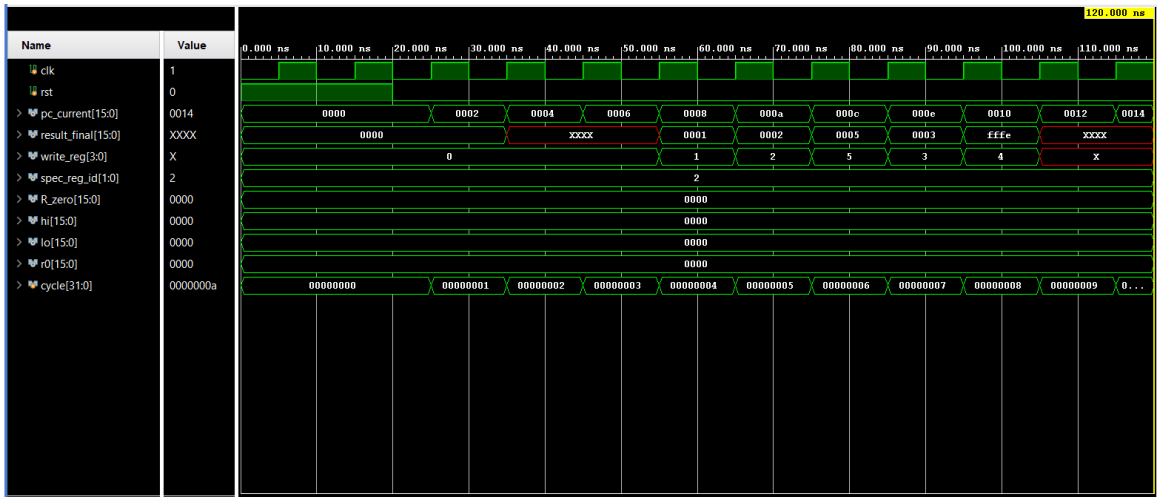
Mục tiêu: Kiểm tra kỹ thuật Forwarding.

Program 8: Test Case Forwarding

```
addi $1, $7, 1        // $1 = 1
addi $2, $7, 2        // $2 = 2
addi $5, $7, 5        // $5 = 5
add $3, $2, $1        // $3 has been credited, currently in EX/MEM
sub $4, $3, $5        // $4 needs $3 immediately at EX
```

Kết quả mong đợi: Không xảy ra Stall. Unit Forwarding chuyển kết quả của lệnh `add` quay lại đầu vào ALU cho lệnh `sub` ngay lập tức.

Kết quả mô phỏng trên Vivado:



Hình 5.14: *Kết quả mô phỏng cho lệnh Data Hazard (RAW).*

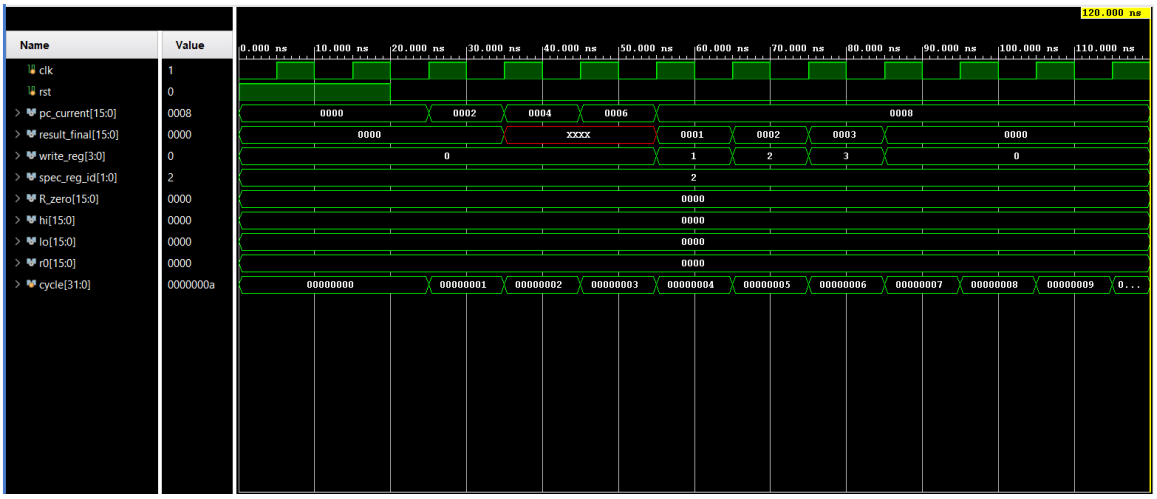
5.1.9 Lệnh Halt

Program 9: Test Case Halt

```
addi $1, $7, 1        // $1 = 1
addi $2, $7, 2        // $2 = 2
add $3, $2, $1        // $3 = 3
hlt                   // Stop the program
addi $5, $7, 5        // Not run
```

Kết quả mong đợi: PC ngừng tăng, tín hiệu Halt tích cực.

Kết quả mô phỏng trên Vivado:



Hình 5.15: *Kết quả mô phỏng cho lệnh Halt.*

5.2 Đánh giá hiệu năng hệ thống

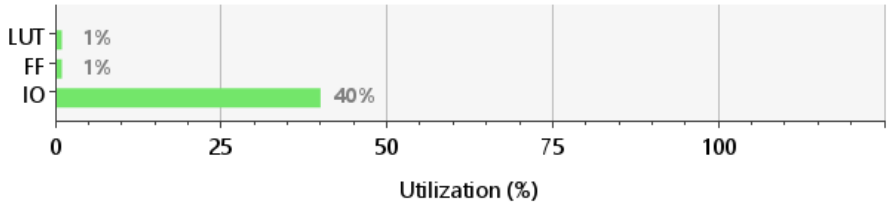
Dựa trên kết quả tổng hợp (Synthesis) và thực thi (Implementation) trên chip xc7z010clg400-1, nhóm đưa ra các đánh giá chi tiết về tài nguyên, thời gian và năng lượng tiêu thụ.

5.2.1 Tài nguyên phần cứng (Resource Utilization)

Kết quả tổng hợp cho thấy thiết kế sử dụng tài nguyên rất khiêm tốn so với dung lượng của FPGA, đảm bảo khả năng mở rộng trong tương lai. Chi tiết được thể hiện trong Bảng 5.16.

Nhận xét và Đánh giá:

Resource	Utilization	Available	Utilization %
LUT	1	17600	0.01
FF	15	35200	0.04
IO	40	100	40.00



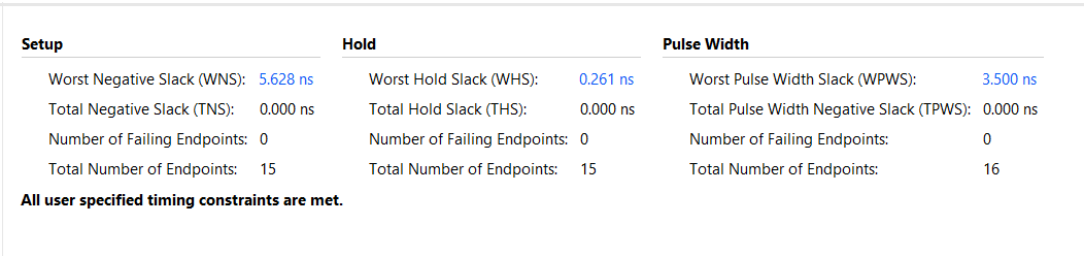
Hình 5.16: *Thống kê tài nguyên sử dụng sau Implementation*

- **Về tài nguyên tính toán (Logic & Registers):** Thiết kế chỉ chiếm tỷ lệ rất nhỏ (dưới 1%) tài nguyên LUTs và Flip-Flops. Điều này chứng tỏ kiến trúc vi xử lý được thiết kế tối ưu, loại bỏ hoàn toàn các logic dư thừa.
- **Về tài nguyên giao tiếp (I/O):** Số lượng cổng I/O sử dụng là 40 chân. Đây là chủ đích thiết kế nhằm phục vụ quá trình kiểm thử và gỡ lỗi. Các tín hiệu nội tại quan trọng như bộ đếm chương trình (`pc_current`) và kết quả tính toán (`result_final`) được định tuyến ra các chân vật lý để quan sát chính xác hành vi của Pipeline. Trong phiên bản thương mại hoặc ứng dụng thực tế, các cổng debug này sẽ được loại bỏ, giúp giảm thiểu đáng kể số lượng chân I/O tiêu thụ.

Kết luận: Kết quả Implementation chứng minh thiết kế đảm bảo tính tối ưu về tài nguyên logic và tính khả thi cao trong việc kiểm thử phần cứng.

5.2.2 Thời gian và Tốc độ (Timing Analysis)

Phân tích định thời (Timing Analysis) cho thấy thiết kế đạt yêu cầu về tần số hoạt động với độ trễ tín hiệu nằm trong giới hạn cho phép.



Hình 5.17: *Báo cáo tổng hợp về Timing Summary*

Dựa trên Hình 5.17:

- **Worst Negative Slack (WNS):** +5.628 ns.
- **Nhận định:** Giá trị WNS dương lớn cho thấy mạch thỏa mãn hoàn toàn các ràng buộc về thời gian (Timing Constraints Met) và có độ dự trữ (Timing Margin) an toàn.
- **Tính toán tần số tối đa thực tế (F_{max}):**

$$T_{min_delay} = T_{clk} - WNS = 8.000 - 5.628 = 2.372 \text{ ns}$$

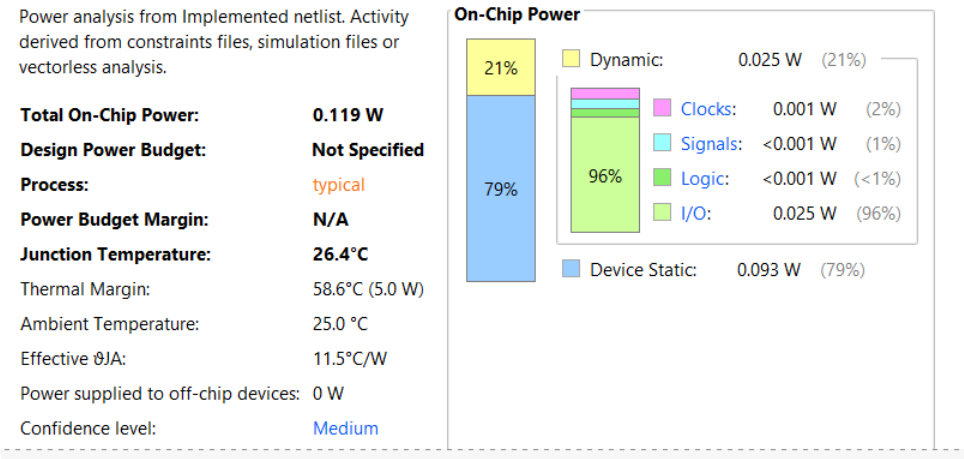
Từ đó, tần số hoạt động tối đa theo lý thuyết của kiến trúc là:

$$F_{max} = \frac{1}{T_{min_delay}} \approx 421.58 \text{ MHz}$$

Kết luận: Với đường dẫn dữ liệu ngắn (chỉ mất khoảng 2.37 ns để xử lý), CPU có khả năng hoạt động ổn định ở tần số cao.

5.2.3 Năng lượng tiêu thụ (Power Consumption)

Ước tính năng lượng được thực hiện trên netlist sau khi Implementation (Hình 5.18).



Hình 5.18: *Báo cáo năng lượng tiêu thụ trên chip*

- **Tổng công suất (Total On-Chip Power):** 0.119 W.
- **Nhiệt độ hoạt động (Junction Temp):** 26.4°C (Ổn định, không gây áp lực tản nhiệt lên hệ thống).

- **Phân bố:** Năng lượng tĩnh (Device Static) chiếm 79% (0.093W), năng lượng động chiếm 21% (0.025W). Mức tiêu thụ thấp do quy mô nhỏ gọn của thiết kế.

5.2.4 Hiệu năng kiến trúc (CPI - Cycles Per Instruction)

Để đánh giá hiệu suất thực tế của kiến trúc Pipeline, nhóm đã thực hiện đo đạc trên kịch bản kiểm thử các lệnh tính toán ALU0.

Kết quả đo đạc từ mô phỏng:

- Tổng số lệnh thực thi (N): 19 lệnh.
- Tổng số chu kỳ clock đo được (C): 23 chu kỳ.

Tính toán chỉ số CPI:

$$CPI = \frac{\text{Tổng số chu kỳ clock}}{\text{Tổng số lệnh}} = \frac{23}{19} \approx 1.21$$

Đánh giá: Kết quả $CPI \approx 1.21$ phản ánh đúng đặc tính của kiến trúc Pipeline (bao gồm độ trễ nạp đầy đường ống ban đầu). Khi số lượng lệnh thực thi tăng lên, chỉ số này sẽ tiệm cận mức lý tưởng ($CPI \approx 1$), ưu việt hơn nhiều so với kiến trúc Đa chu kỳ (Multi-cycle) vốn tốn trung bình 3-5 chu kỳ cho mỗi lệnh.

Kết luận: Các số liệu thực nghiệm đã cho thấy logic điều khiển đường ống (Pipeline Controller) và cơ chế xử lý xung đột (Hazard Unit) giúp đảm bảo hệ thống hoạt động ổn định và đúng đắn, nâng cao hiệu suất hoạt động.

6 Tổng kết

6.1 Vivado Project Source Code

Mã nguồn của đồ án có thể truy cập tại đây.

6.2 Kết luận

Trong khuôn khổ đồ án môn học *Đồ án Thiết kế Luận lý*, nhóm đã nghiên cứu, thiết kế và hoàn thiện một bộ xử lý *RISC CPU 16-bit* dựa trên kiến trúc *MIPS*. Hệ thống được xây dựng theo mô hình pipeline, bao gồm các khối chức năng chính như *Instruction Fetch*, *Decode*, *Execute*, *Memory* và *Write Back*, đảm bảo thực hiện đúng luồng xử lý lệnh theo thiết kế đề ra.

Trong quá trình thực hiện, nhóm đã tiến hành kiểm thử từng module riêng lẻ cũng như kiểm thử tích hợp toàn hệ thống thông qua mô phỏng. Các kết quả mô phỏng cho thấy CPU hoạt động đúng với đặc tả, thực hiện chính xác các nhóm lệnh đã hỗ trợ và xử lý được các tình huống cơ bản liên quan đến pipeline và điều khiển luồng chương trình.

Thông qua đồ án, nhóm đã có cơ hội để củng cố và mở rộng những kiến thức và kỹ năng như:
Đầu tiên, về mặt kiến thức chuyên môn:

- Đồ án giúp nhóm áp dụng các kiến thức lý thuyết từ các môn học vào thực tế.
  - Môn Thiết kế luận lý với HDL: Vận dụng kiến thức mạch luận lý để thiết kế các module, áp dụng được các kỹ thuật mô tả phần cứng bằng ngôn ngữ Verilog HDL, hiểu rõ hơn về timing, clock và đồng bộ hóa trong hệ thống số và tối ưu các thiết kế khối logic.
  - Môn Kiến trúc máy tính: Nắm vững kiến trúc thiết kế máy tính và hiểu sâu hơn về các vấn đề xung đột dữ liệu hay phần cứng và đề xuất các phương pháp giải quyết.
  - Ngoài ra, nhóm còn áp dụng được những kỹ năng đã học ở những môn học khác để bổ trợ cho đồ án như các kỹ năng vẽ diagram...

Tiếp theo, về kỹ năng thực hành, nhóm đã thành thạo hơn trong việc sử dụng Verilog để mô tả phần cứng và thiết kế testbench để kiểm tra các module. Cuối cùng, về kỹ năng mềm, nhóm đã tích lũy và phát triển được một số kỹ năng như kỹ năng teamwork, kỹ năng tìm hiểu và nghiên cứu tài liệu, kỹ năng quản lý dữ án,..

6.3 Hạn chế và hướng phát triển

Trong quá trình thực hiện đồ án, nhóm cũng gặp phải những khó khăn nhất định như gặp khó khăn trong việc hiểu rõ các datapath của CPU, debugging trong pipeline, tích hợp các module từ các thành viên trong nhóm và xử lý các trường hợp nhóm lệnh đặc biệt của special register. Song, nhóm cũng đã rút được các bài học kinh nghiệm để hoàn thành đồ án và tích lũy cho các đồ án và công việc sau này. Mặc dù đã đạt được những kết quả nhất định nhưng nhóm nhận thấy đồ án vẫn còn một số hạn chế cần khắc phục và cải tiến như:

- **Hạn chế trong thiết kế kiến trúc:** Thiết kế tổng thể của hệ thống chưa thực sự tối ưu, thể hiện qua số lượng tín hiệu I/O còn khá lớn. Một số module, điển hình như `EX_Stage`, có nhiều cổng vào/ra, gây khó khăn trong quá trình tích hợp. Bên cạnh đó, việc thiết kế và triển khai cơ chế xử lý *hazard* và *control flow* còn phức tạp; xử lý *flush* bị phân tán ở nhiều module khác nhau. Đối với nhóm lệnh *branch* và *jump*, chưa có sự phân cấp rõ ràng về mức độ ưu tiên, dẫn đến việc tính toán và xử lý ở các giai đoạn pipeline khác nhau, gây thiếu nhất quán trong thiết kế.
- **Hạn chế về chức năng:** Hệ thống chưa được mở rộng thêm tập lệnh, do đó khi so sánh với kiến trúc MIPS tiêu chuẩn, CPU chưa hỗ trợ đầy đủ các nhóm lệnh cần thiết.
- **Hạn chế trong triển khai thực tế:** Thiết kế hiện tại mới dừng lại ở mức mô phỏng và chưa được triển khai trên phần cứng thực. Vì vậy, các vấn đề liên quan đến tài nguyên, timing và hiệu năng thực tế của hệ thống chưa được đánh giá đầy đủ.
- **Hạn chế về khả năng mở rộng:** Nhiều tín hiệu trong thiết kế hiện tại được *hardcode*, gây khó khăn cho việc mở rộng, tái sử dụng và nâng cấp hệ thống trong các giai đoạn phát triển tiếp theo.

Sau khi nhìn nhận các nhược điểm của đồ án, nhóm thống nhất đề xuất một số hướng cải tiến và phát triển hệ thống trong tương lai như sau:

- **Tối ưu hóa thiết kế và tổ chức module:** Chuẩn hóa lại giao tiếp giữa các module theo hướng gom nhóm tín hiệu, giảm số lượng cổng I/O không cần thiết. Các khối chức năng lớn như EX_Stage hoặc Pipeline Registers có thể được tái cấu trúc nhằm tăng tính module hóa, dễ tích hợp và thuận tiện cho việc mở rộng, tái sử dụng trong các thiết kế sau này.
- **Cải tiến Pipeline Control và xử lý Hazard:** Thiết kế lại khối điều khiển pipeline theo hướng tập trung, hạn chế việc xử lý flush và stall bị phân tán ở nhiều module khác nhau. Bổ sung cơ chế phân cấp rõ ràng cho các nhóm lệnh branch và jump nhằm đảm bảo tính nhất quán trong việc tính toán địa chỉ và xử lý control hazard. Ngoài ra, có thể nghiên cứu áp dụng các kỹ thuật forwarding và hazard detection nâng cao để cải thiện hiệu năng pipeline.

- **Mở rộng Instruction Set:** Bổ sung thêm các lệnh trong tập lệnh MIPS tiêu chuẩn như các lệnh logic, shift, load/store nâng cao hoặc các lệnh thao tác với special registers. Việc mở rộng tập lệnh giúp hệ thống tiệm cận hơn với kiến trúc MIPS thực tế và tăng khả năng ứng dụng của CPU.
- **Triển khai và đánh giá trên phần cứng thực:** Thực hiện tổng hợp (synthesis) và triển khai thiết kế trên FPGA nhằm đánh giá chính xác các thông số về thời gian, tài nguyên và hiệu năng thực tế. Quá trình này cũng giúp phát hiện các vấn đề không xuất hiện trong mô phỏng và là bước quan trọng để hoàn thiện hệ thống.
- **Giảm hardcode và tăng khả năng mở rộng:** Thay thế các tín hiệu và tham số được hardcode bằng các tham số cấu hình (parameter/generic) để tăng khả năng scale và reuse. Điều này cho phép dễ dàng điều chỉnh độ rộng dữ liệu, số stage pipeline hoặc mở rộng kiến trúc trong các phiên bản phát triển tiếp theo.

Lời cảm ơn

Nhóm xin gửi lời cảm ơn chân thành đến giảng viên **Tôn Huỳnh Long** đã tận tình góp ý và hỗ trợ nhóm trong suốt quá trình thực hiện đồ án môn học **Đồ án Thiết kế Luận lý**. Những kiến thức chuyên môn, kinh nghiệm thực tiễn và các nhận xét của thầy là nền tảng quan trọng giúp nhóm định hướng đúng đắn và hoàn thành đề tài *Thiết kế RISC CPU 16-bit dựa trên kiến trúc MIPS*.

Nhóm cũng xin cảm ơn Khoa Khoa học và Kỹ thuật Máy tính cùng Nhà trường đã tạo điều kiện về cơ sở vật chất, tài liệu học tập và môi trường nghiên cứu thuận lợi để nhóm có thể hoàn thành đồ án. Thông qua quá trình thực hiện đề tài, nhóm đã tích lũy thêm nhiều kiến thức và kinh nghiệm quý báu, làm cơ sở cho các học phần và công việc nghiên cứu, phát triển trong tương lai.

Tài liệu tham khảo

[1] Slide bài giảng môn *Kiến trúc máy tính*, Khoa Khoa học và Kỹ thuật Máy tính, Trường Đại học Bách Khoa TP.HCM, năm học 2024–2025.

[2] Slide bài giảng môn *Thiết kế luận lý với HDL*, Khoa Khoa học và Kỹ thuật Máy tính, Trường Đại học Bách Khoa TP.HCM, năm học 2023–2024.

[3] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, 5th Edition, Morgan Kaufmann, 2014.

[4] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th Edition, Morgan Kaufmann, 2019.